

PEP-AI Validated Quantization for Energy-Efficient Perception in Intelligent Transportation Systems

Tamás Menyhárt^{1*}, András Hajdu¹ and Róbert Lakatos¹

^{1*}Department of Data Science and Visualization, Faculty of
Informatics, University of Debrecen, Debrecen, Hungary.

*Corresponding author(s). E-mail(s): menyhart.tamas@inf.unideb.hu;
Contributing authors: hajdu.andras@inf.unideb.hu;
lakatos.robert@inf.unideb.hu;

Abstract

Background: Post-training INT8 quantization reduces the inference energy of camera-based perception, but quantized models are judged by output accuracy alone, which hides where the error arises and how it propagates.

Methods: Five pretrained detection and classification networks were quantized without retraining into FP16, INT8 and FP8 TensorRT engines on an NVIDIA H200. We prove noise and propagation bounds, test them on layer-wise activations, measure accuracy, energy and robustness, and validate a static analysis.

Results: INT8 was 1.6–11.8× faster than FP32 and used 72–91% less energy; FP16 was lossless, FP8 lost at most 0.4 points and INT8 0.25 mAP to 41 top-1 points. Noise propagation, not network size, explained this range: one measurement of the quantized head input predicted the relative INT8 loss within a factor of about two (tenfold per 12 dB). From the FP32 parameters alone, the distribution-independent FP8 noise was predicted within 0.14 dB, the distribution-dependent INT8 noise only within 1–6 dB. INT8 also lost up to 36% under low contrast and 17 points with the calibration order, and broke the duplicate suppression of an NMS-free detector; FP8 did not. Keeping 20 ranked layers in FP16 restored EfficientNet-B0 from 24.8% to 66.0%.

Conclusion: Quantization error is predictable from FP32 statistics up to a measurable propagation factor, which decides whether INT8 is safe; FP8 is the distribution-independent alternative. For 1 000 vehicles, INT8 saves up to 1 GWh and 251 t CO₂ per year.

Keywords: post-training quantization, INT8, FP8, TensorRT, energy efficiency, explainable AI, sustainable AI, intelligent transportation systems

1 Introduction

Camera-based perception never pauses: it runs on every camera stream of an automated vehicle for as long as the vehicle operates, so its energy grows with fleet size, frame rate and operating hours [1]. If automated driving is widely adopted, the emissions of onboard computing could approach those of today’s data centres [2]. The energy cost of machine learning is already recognised as a sustainability problem [3–6], and for many deployed models it is inference, not training, that dominates [7, 8]; reported savings should therefore be measured rather than estimated [9]. In transport, the European Green Deal asks for a 90% cut of emissions by 2050, at the same time as automated mobility is to be deployed at scale [10, 11]. Choosing the numerical format of a perception network thus means trading energy cost and emissions against perception errors. In AI-based business optimisation (AIBO), model outputs feed managerial decisions whose cost includes both operating expenses and the consequences of errors [12]. For quantization, such a decision needs two numbers that aggregate accuracy does not give: the measured energy per inference and the error rate relative to the reference model.

Post-training quantization (PTQ) converts the weights and activations of a trained network from 32-bit floating point (FP32) to 8-bit integers (INT8), without retraining [13–16]. This cuts memory traffic fourfold, and an 8-bit integer multiplication or addition costs 18–30 times less energy than its FP32 counterpart [17]. On a GPU, however, these savings only materialise with integer kernels generated by an inference compiler [18, 19]; quantization that is merely simulated in floating point saves nothing. Most research on PTQ aims at minimising the output error of the quantized model. Adaptive rounding [20], layer-wise optimisation on small calibration sets [21] and block reconstruction [22, 23] optimise how the weights are rounded; cross-layer equalisation balances per-channel ranges [24]; outlier channel splitting reduces clipping [25]; and mixed-precision methods allocate bit widths from second-order sensitivities [26, 27].

On the analytical side, the rounding error of a single quantizer is usually treated as additive noise that is uniform within the clipping range. This model underlies analytical range setting [15, 28] and the comparison of integer and floating-point formats [29, 30]. How the noise of many quantizers combines as it passes through a nonlinear network has been bounded under distributional assumptions [31] or kept in check by non-expansive layers [32]. What is missing is a decomposition into quantities that can be estimated from FP32 statistics and then tested on a deployed engine. With such a decomposition, one could tell before deployment which networks tolerate INT8 and which layers have to stay in higher precision.

In practice, PTQ methods are judged by aggregate accuracy. Aggregate accuracy can stay stable while the internal representations change, and compressed models can disagree with the original on a subset of inputs [33]. It does not show where the error arises, whether it predicts the errors that matter, or how it behaves under

distribution shift, and these are exactly the questions that safety standards for road vehicles require to be analysed [34–36]. Some pieces are known: depthwise-separable networks are sensitive to per-tensor activation quantization [18, 37], and quantized classifiers are more sensitive to natural corruptions such as fog or blur [38–40]. For detectors running on integer kernels, however, nobody has yet quantified how the error of individual layers relates to detection failures.

In our conference paper we introduced the Precise, Explainable and Provable (PEP-AI) framework for validating quantized networks at the level of their activations [41]. It consisted of the median relative error of channel-projected activations, symmetric-logarithmic hexbin densities and an attenuation model of layer-wise error propagation. That paper analysed a single backbone in simulated INT8 and showed qualitative failure examples; deployed integer engines, other architectures and a quantitative link between the deviation and detection errors were left for future work. The present article takes up these open points:

- *Theory*: a rounding-noise lemma for INT8 and floating-point formats, a corollary for static scales, an additive decomposition of the head-input error over the quantizers and a bound for contracting chains, with proofs and quantitative tests (Sections 2.5 and 2.6).
- *Measurement*: strict-FP32, FP16, INT8 and FP8 TensorRT engines of five networks on an H200, with accuracy and paired bootstrap intervals, latency, jitter, memory traffic and energy.
- *Validation*: a detection-level test of the deviation–error link, robustness under eight adverse conditions, and two INT8 failure modes that aggregate accuracy does not reveal: dependence on the order of the calibration images and broken duplicate suppression in an NMS-free detector.
- *Prediction*: a five-step procedure that screens a network for INT8 from FP32 statistics and, after one measurement of the quantized head input, predicts its relative accuracy loss, worked through and checked against the measurements for every step (Section 3.8).
- *Static analysis*: a data-free analysis of the FP32 weights and batch-normalisation parameters that estimates the INT8 and FP8 noise of any network without executing it, validated on two networks not used elsewhere (Sections 2.13 and 3.9).
- *Decision*: a fleet cost model with a break-even penalty per quantization-induced error.

The complete pipeline, the static analysis tool and all results are openly available at <https://github.com/EgyipTomi425/EchteAI>, and every number, table and figure of this article can be regenerated from it. The step-by-step proofs, implementation details and supplementary analyses are given in Additional file 1 (Appendices A and B), and the extended tables and figures, numbered C1–C18 and C1–C14, in its Appendix C.

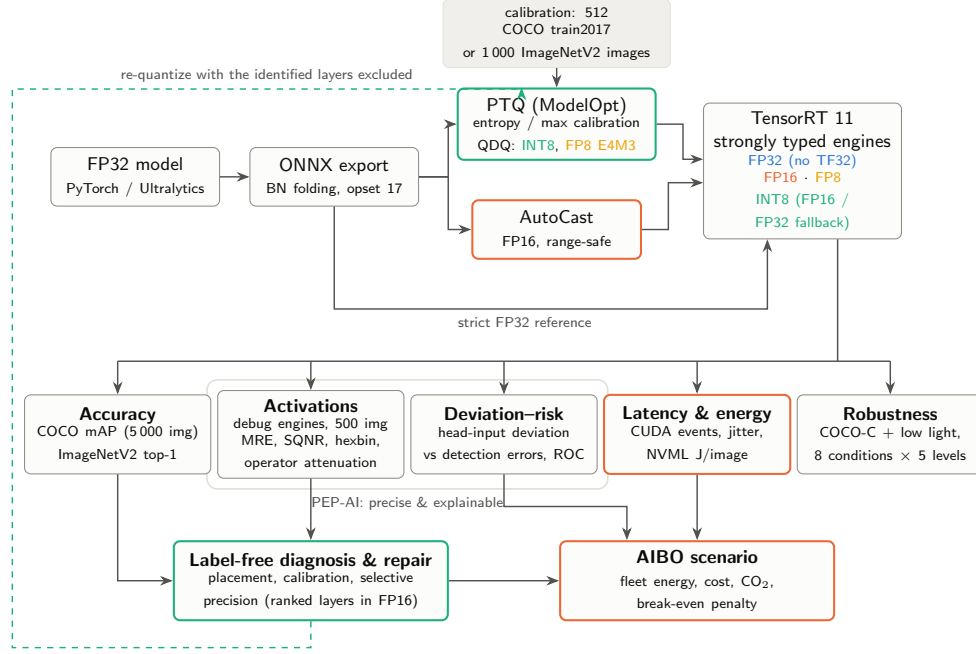


Fig. 1 PEP-AI validation pipeline. The FP32 model is exported once and turned into an FP16 variant and into INT8 and FP8 variants that share one calibration; every variant is compiled into a strongly typed TensorRT engine and evaluated along five measurement branches. The activation analysis feeds back into quantization (label-free diagnosis and repair, dashed), and the measured energy, accuracy and error rates feed the AIBO scenario.

2 Methods

2.1 PEP-AI validation pipeline

PEP-AI compares the activations of a quantized network with those of its FP32 reference. The three letters stand for three requirements. *Precise* means per-tensor, per-image deviation statistics for every analysed layer of the deployed engine. *Explainable* means an analytical model of where the deviation is injected and how it propagates, supported by activation maps and density plots. *Provable* means that statements are either proved under explicit assumptions (Lemma 1, Corollary 2, Propositions 3 and 4) or reported as empirical claims with a stated sample and uncertainty. The pipeline (Fig. 1) exports each FP32 model to ONNX, quantizes it into explicit quantize-dequantize (QDQ) graphs and compiles strongly typed TensorRT engines. It then measures accuracy, activation deviation, the link between deviation and errors, latency, energy and robustness, and the layer-level diagnosis is fed back into quantization. Every stage is scripted and seeded, so that the pipeline regenerates every number, figure and table of this article.

2.2 Models and data

We evaluated five convolutional architectures that cover two-stage detection, one-stage detection and classification (Table 1). The two-stage detector is Faster R-CNN with a ResNet-50 FPN backbone [42], with the torchvision COCO weights and its frozen batch normalisation folded into the convolutions before export. The one-stage detectors are YOLOv10-S and YOLOv10-X [43] with their NMS-free one-to-one head. The classifiers are EfficientNet-B0 [44] and DenseNet-121 [45], both with torchvision ImageNet weights.

Table 1 Evaluated architectures

Model	Task	Input	Params (M)	GFLOPs	Convs	Weights (MB)		
						FP32	FP16	INT8
Faster R-CNN R50-FPN	det.	800×1088	26.8	240.4	61	107.3	53.7	27.0
YOLOv10-S	det.	640×640	8.1	24.9	86	29.0	14.5	7.4
YOLOv10-X	det.	640×640	31.8	170.8	174	117.9	58.9	29.9
EfficientNet-B0	cls.	224×224	5.3	0.8	81	21.1	10.5	6.8
DenseNet-121	cls.	224×224	8.0	5.7	120	32.2	16.1	9.3

Faster R-CNN: backbone and FPN only (the part that is quantized); GFLOPs at the benchmark input size. INT8 weight size counts quantized weights at 1 byte and the remaining tensors in FP16.

The detectors were evaluated on the complete COCO 2017 validation split (5 000 images) [46]; the activation analysis and the robustness study use the first 500 images of a seeded permutation of it. For calibration we took 512 seeded images from the COCO 2017 *training* split, so that calibration and evaluation data never overlap. The classifiers were evaluated on ImageNetV2 (matched frequency) [47], which we split with a fixed seed into 1 000 calibration and 9 000 evaluation images. Pre-processing follows the reference implementation of each model (Table 2). About 40% of all COCO objects belong to traffic-relevant categories.

Table 2 Data splits

Split	Purpose	Images	Objects	Obj./img	S/M/L (%)	Traffic (%)
train2017 subset	calibration (detectors)	512	3 401	6.6	43 / 32 / 25	44
val2017	mAP, deviation-risk	5 000	36 335	7.3	42 / 34 / 24	41
val2017 first 500	activations, robustness	500	3 787	7.6	44 / 33 / 23	41
INV2 calibration	calibration (classifiers)	1 000	—	—	—	—
INV2 evaluation	top-1, activations	9 000	—	—	—	—

Traffic: share of objects in the classes person, bicycle, car, motorcycle, bus, truck, traffic light and stop sign. S/M/L: COCO size classes by box area ($< 32^2$, 32^2 – 96^2 , $> 96^2$ pixels). All subsets are drawn with a fixed seed.

2.3 Quantization and deployment

Post-training quantization used NVIDIA TensorRT Model Optimizer [48] without fine-tuning [13–15]. For a tensor x and a scale $s > 0$, the symmetric INT8 quantizer is

$$Q_s(x) = s \cdot \text{clip}(\lfloor x/s \rfloor, -127, 127), \quad (1)$$

with one scale per output channel for weights and one scale $s = \alpha/127$ per tensor for activations. The clipping range α is set on the calibration images by entropy calibration [49] (Eq. (A1)) or, as an ablation, by the observed maximum. Small calibration sets can make the result depend on the sample [21], so we recalibrated every model on seeded random subsets of its calibration set and, as a control, with an order-independent implementation of entropy calibration (Additional file 1, Section A.1). The FP8 variants [50] use $s = \alpha/448$, with α set to the observed maximum (the toolchain default for FP8), and round to the nearest E4M3 number; they use the same calibration data and the same exclusions. FP16 was produced by automatic mixed precision [51]. From one calibration we derive two INT8 graphs that share every scale: *INT8*, in which operations without QDQ nodes run in FP16 (this is the deployed variant), and *INT8 (FP32 fallback)*, which isolates the effect of quantization in the activation analysis. Decoding and post-processing stay in high precision. Quantizing the TopK of the YOLOv10 head would restrict the class scores to 127 levels, and quantizing the distribution-focal-loss (DFL) box decoder distorts large objects; we report both of these deployment artefacts as ablations (Fig. C1). INT4 was not evaluated, because the H200 has no INT4 tensor-core arithmetic and TensorRT supports INT4 only for the weights of matrix multiplications [52, 53].

All variants were compiled with TensorRT 11.3 [19] into *strongly typed* engines, in which the precision of every layer follows the graph. We disabled TF32, so that the FP32 baseline is true single precision. Static-shape engines were built for batch sizes 1, 6 and 8. For evaluation, Faster R-CNN uses a dynamic-shape engine (input sides 256–1344), while its proposal network and ROI heads run in FP32 PyTorch. We read back from the engine inspector the precision in which every convolution is actually executed. The environment is listed in Table 3, and implementation details are given in Additional file 1, Section A.1.

Table 3 Hardware and software environment

Component	Version / specification
GPU	NVIDIA H200, power limit 700 W
GPU driver	590.48.01
CPU	Intel Xeon Platinum 8480C
TensorRT	11.3.0.99
NVIDIA ModelOpt	0.47.0
ONNX Runtime	1.24.4
PyTorch / CUDA	2.11.0+cu128 / 12.8

2.4 Activation-level deviation metrics

We mark every output tensor of the analysed operators (convolutions, activation functions, additions, pooling, concatenation, resizing and non-foldable batch normalisation) as an engine output and compare the strict-FP32 engine with the INT8 (FP32 fallback) engine on identical inputs. Marking outputs prevents some fusions, but it leaves the quantized arithmetic unchanged. With f an FP32 tensor, q its quantized counterpart and $e = q - f$, the metrics of Table 4 are

$$\text{MRE}_{\text{elem}} = \text{median}_i \frac{|e_i|}{|f_i| + \eta}, \quad (2)$$

$$A(x, y) = \max_c F(c, x, y), \quad \text{MRE}_{\text{proj}} = \text{median}_{x, y} \frac{|A_q(x, y) - A_f(x, y)|}{|A_f(x, y)| + \eta}, \quad (3)$$

$$r = \frac{\|e\|_2}{\|f\|_2}, \quad \text{SQNR} = 10 \log_{10} \frac{\|f\|_2^2}{\|e\|_2^2} \text{ dB}, \quad (4)$$

with $\eta = 10^{-6}$. To give a feeling for the scale: an SQNR of 40, 20 and 0 dB corresponds to a relative error r of 1%, 10% and 100%, and every 6 dB is a factor of two in r . Task accuracy is the COCO box mAP@[.5:.95] for the detectors [46, 54], complemented for the road-user classes by the AP at IoU 0.5 and by recall and precision at the operating threshold of 0.5, and the top-1 accuracy for the classifiers. MRE_{proj} is the metric of the conference paper; r and SQNR are scale-free and standard in the quantization literature [15]. Per-image values are aggregated over the 500 analysis images by their median and interquartile range. By *head-input* tensors we mean the tensors consumed by the task head: the five FPN levels, the three YOLOv10 head inputs and the last feature map of the classifiers. For the detectors, we also compute MRE_{proj} separately inside and outside the ground-truth boxes.

2.5 Quantization noise model

To interpret the deviations we use the additive noise model that also underlies analytical range setting [15, 16, 28]. It rests on two assumptions. (A1) Inside the clipping range, the INT8 rounding error $e = Q_s(x) - x$ is uniform on $[-s/2, s/2]$ and uncorrelated with x ; this holds when s is small compared with the spread of x . (A2) Under the power-weighted measure $x^2 dP(x)$, the significand $m = |x|/2^{\lfloor \log_2 |x| \rfloor}$ of an activation is log-uniform on $[1, 2)$; this holds approximately for any distribution whose logarithm spreads over several octaves.

Lemma 1 (Rounding noise) *Let x be an activation with root-mean-square value σ and normalised range $\kappa = \alpha/\sigma$. Under (A1), the INT8 quantizer of Eq. (1) with $s = \alpha/127$ has the error power*

$$\mathbb{E} e^2 = \frac{s^2}{12} \Pr(|x| \leq \alpha) + \mathbb{E} \left[(|x| - \alpha)^2 \mathbb{I}\{|x| > \alpha\} \right], \quad (5)$$

and, without clipping,

$$\text{SQNR}_{\text{INT8}} = 10 \log_{10} (12 \cdot 127^2) - 20 \log_{10} \kappa \approx 52.9 \text{ dB} - 20 \log_{10} \kappa. \quad (6)$$

Table 4 PEP-AI metrics. f : FP32 tensor, q : quantized tensor, $e = q - f$, A : channel-wise maximum projection.

Metric	Definition	PEP-AI role	Interpretation
MRE_{proj}	Eq. (3)	precise, explainable	conference-paper metric; spatial, on the dominant response per location
MRE_{elem}	Eq. (2)	precise	element-wise median relative error over all channels
r	$\ e\ _2/\ f\ _2$	precise	scale-free relative error; $r \geq 1$ means no usable signal
SQNR	$-20 \log_{10} r$	precise	signal-to-quantization-noise ratio in dB
a_n	$r_{\text{out}}/\max r_{\text{in}}$	explainable	operator-level attenuation (< 1) or amplification (> 1)
s_n	SQNR drop across node n	explainable	error injected by a layer; drives selective quantization
AUC	ROC of deviation vs. errors	provable	how well internal deviation predicts detection errors

Under (A2), round-to-nearest in a floating-point format with p significant bits has, in its normal range, the error power $\mathbb{E} e^2 = \mathbb{E} x^2 2^{-2p}/(8 \ln 2)$, so that, independently of κ ,

$$\text{SQNR}_{\text{FP}} = 20p \log_{10} 2 + 10 \log_{10} (8 \ln 2) \approx 6.02p + 7.44 \text{ dB}. \quad (7)$$

The idea behind both results is simple. INT8 has a fixed grid with step $s = \alpha/127$, and a rounding error that is uniform over one step has the mean square $s^2/12$; dividing the signal power σ^2 by it leaves only the ratio $\kappa = \alpha/\sigma$. A floating-point grid, by contrast, is spaced in proportion to the value itself, so the relative error is the same for small and large values and κ drops out; averaging over where a value falls within its octave gives the constant $8 \ln 2$. For E4M3 ($p = 4$), Eq. (7) gives 31.5 dB, and for FP16 ($p = 11$) 73.7 dB. The two formats behave very differently: the relative precision of a floating-point format does not depend on the calibrated range [29, 30], whereas INT8 loses 20 dB per decade of κ . Eq. (5) expresses the familiar trade-off between the granular noise of a wide range and the clipping of a narrow one [25, 28]. A Monte Carlo simulation reproduces Eqs. (5)–(7) within 0.1 dB (Fig. 2), and the proofs are given in Additional file 1, Section A.2. Because TensorRT uses static scales, Lemma 1 also tells what happens when the signal level changes after calibration.

Corollary 2 (Static scales) *Let the scale s be calibrated for an activation with root-mean-square value σ_{cal} , and let the activation at inference be c times larger in distribution, $c > 0$. If no value is clipped and (A1), (A2) hold, then*

$$\Delta \text{SQNR}_{\text{INT8}} = 20 \log_{10} c, \quad \Delta \text{SQNR}_{\text{FP8}} = 0 \quad \text{for } c \sigma_{\text{cal}} \gg 2^{-6} s, \quad (8)$$

where $2^{-6} s$ is the smallest normal E4M3 number.

In words: INT8 loses 6 dB each time the signal is halved, while FP8 loses nothing. A signal that grows ($c > 1$) enters the clipping regime of Eq. (5), which the tight INT8

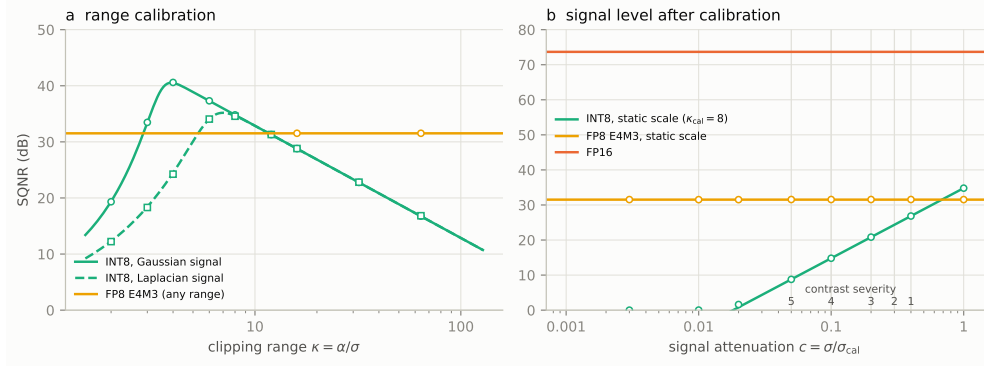


Fig. 2 Quantization noise model (lines: Eqs. (5)–(7); markers: Monte Carlo simulation with 4×10^5 samples). **a** SQNR of INT8 versus the normalised clipping range $\kappa = \alpha/\sigma$ for Gaussian and Laplacian signals, and of FP8 E4M3. **b** SQNR of a signal attenuated by the factor c after the scale was calibrated at $\kappa_{\text{cal}} = 8$ (static per-tensor scale); vertical lines: contrast factors of the five COCO-C contrast severities.

range cannot absorb, unlike the maximum-calibrated FP8 range. Reduced contrast and fog are shifts of this kind. The attenuation c has to be measured at the quantizers, however, because biases and mean responses do not scale with the pixel contrast (Additional file 1, Section A.2).

2.6 Error propagation model

The conference paper modelled the propagation of quantization error between layers l and $l + 1$ as

$$\epsilon^{(l+1)} = \mathcal{A} \left(\mathcal{F}(y_f^{(l)} + \epsilon^{(l)}) - \mathcal{F}(y_f^{(l)}) \right) + \delta^{(l+1)}, \quad (9)$$

with layer transformation $\mathcal{F}$, local rounding noise $\delta^{(l+1)}$ and an attenuation operator $\mathcal{A}$. Other analytical treatments propagate quantization noise under distributional assumptions [31] or constrain the layers to be non-expansive [32]. Here we linearise every node n around its FP32 operating point. With FP32 output x_n , deviation e_n , inputs $\pi(n)$, Jacobian J_n and injected rounding noise δ_n ($\delta_n = 0$ for nodes without quantizers),

$$e_n = J_n e_{\pi(n)} + \delta_n, \quad (10)$$

where $e_{\pi(n)}$ stacks the deviations of all inputs. Squared norms of deviations are understood as expectations over the rounding noise, $\|e\|^2 := \mathbb{E} \|e\|^2$. If δ_n is uncorrelated with the propagated deviation, the cross term $2 \mathbb{E} \langle J_n e_{\pi(n)}, \delta_n \rangle$ vanishes, and the relative error $r_n = \|e_n\|/\|x_n\|$ obeys

$$r_n^2 = g_n^2 r_{\pi(n)}^2 + \rho_n^2, \quad g_n = \frac{\|J_n e_{\pi(n)}\|}{\|e_{\pi(n)}\|} \cdot \frac{\|x_{\pi(n)}\|}{\|x_n\|}, \quad \rho_n = \frac{\|\delta_n\|}{\|x_n\|}. \quad (11)$$

The propagation gain g_n is the attenuation operator $\mathcal{A}$ in scalar form. For every analysed node we measure the amplification ratio

$$a_n = \frac{r_n}{\max_{m \in \pi(n)} r_m} \leq \sqrt{g_n^2 + \rho_n^2 / r_{\pi(n)}^2}, \quad (12)$$

with equality for single-input nodes, because $r_{\pi(n)} \leq \max_m r_m$ for stacked inputs; for single-input operators without quantizers, $a_n = g_n$. Additional file 1, Section A.3 derives closed forms of g_n for sigmoid gates, max pooling and non-folded batch normalisation (Eq. (A3)) and proves the following two statements.

Proposition 3 *Assume the linearisation of Eq. (10) and zero-mean, mutually uncorrelated injected noises δ_n . Then the relative error at the head input h is*

$$r_h^2 = \sum_n \Gamma_{n \rightarrow h} \rho_n^2, \quad \Gamma_{n \rightarrow h} = \frac{\mathbb{E} \|J_{n \rightarrow h} \delta_n\|^2}{\mathbb{E} \|\delta_n\|^2} \cdot \frac{\|x_n\|^2}{\|x_h\|^2}, \quad (13)$$

where $J_{n \rightarrow h}$ is the Jacobian from node n to h . If a set S of nodes is kept in FP16, whose injected noise is negligible by Eq. (7), then $r_h^2(S) = r_h^2 - \sum_{n \in S} \Gamma_{n \rightarrow h} \rho_n^2$, and a set of k nodes that minimises $r_h^2(S)$ consists of the k largest contributions $\Gamma_{n \rightarrow h} \rho_n^2$.

Proposition 4 *Let a chain of nodes $1, \dots, n$ satisfy Eq. (11) with gains g_m and injected noise ρ_m . Then*

$$r_n^2 = r_0^2 \prod_{m=1}^n g_m^2 + \sum_{k=1}^n \rho_k^2 \prod_{m=k+1}^n g_m^2. \quad (14)$$

If $g_m \leq \bar{g} < 1$ and $\rho_m \leq \bar{\rho}$ for all m , the relative error stays bounded by $\max\{r_0, \bar{\rho}/\sqrt{1 - \bar{g}^2}\}$ at any depth, and for constant $g_m = g < 1$ and $\rho_m = \rho$ it converges to the fixed point

$$r_\infty = \frac{\rho}{\sqrt{1 - g^2}}, \quad \text{SQNR}_\infty = -20 \log_{10} \rho + 10 \log_{10} (1 - g^2). \quad (15)$$

If instead $g_m \geq g > 1$, the propagated part grows at least as $g^{2n} r_0^2$.

Proposition 4 puts the attenuation effect of the conference paper on a formal footing. The error does not have to shrink from layer to layer; it is enough that its propagated part contracts ($g < 1$), and the error then settles on a plateau r_∞ that is set by ρ and g . Proposition 3 motivates selective precision, and it also suggests how the head-input fidelity can be predicted from FP32 statistics alone. Applying Eq. (1) with the calibrated scales to the FP32 activations gives the injected noise ρ_n^2 of every quantizer exactly, and with unit propagation factors we obtain

$$\text{SQNR}_{\text{add}} = -10 \log_{10} \sum_n \rho_n^2, \quad \bar{\Gamma} = \frac{r_h^2}{\sum_n \rho_n^2} = 10^{(\text{SQNR}_{\text{add}} - \text{SQNR}_h)/10}, \quad (16)$$

where SQNR_h is the measured head-input SQNR. By Proposition 3, $\bar{\Gamma} = \sum_n \Gamma_{n \rightarrow h} \rho_n^2 / \sum_n \rho_n^2$ is the mean propagation factor, weighted by the injected noise. It is below one for a network that attenuates the injected noise and above one for a network that amplifies it (Fig. C2).

2.7 Deviation–risk analysis

We run both engines on all 5 000 validation images with the head-input tensors exposed and compare them at a score threshold of 0.5. As in the conference paper, an image contains a *detection error* if INT8 misses a ground-truth object that FP32 found, or if an FP32 detection has no INT8 counterpart of the same class with $\text{IoU} \geq 0.5$. A *score-tolerant* variant accepts INT8 counterparts down to a score of 0.3; an FP32 detection that still has no counterpart has then genuinely vanished. At the image level, we compare the ROC AUC of the head-input MRE_{proj} with that of a trivial covariate, the number of FP32 detections. For the local deviation defined below, we derive an architecture-specific threshold from the data as the cut that maximises Youden’s index $J = \text{TPR} - \text{FPR}$ for vanishing detections. At the detection level, the *local deviation* d_j of detection j is the median MRE_{proj} of the head-input maps inside its box, and it enters the logistic model

$$\text{logit } \Pr(v_j = 1) = \beta_0 + \beta_1 \log_2 d_j + \beta_2 m_j + \beta_3 \log A_j \quad (17)$$

for the vanishing v_j of the detection, with score margin $m_j = -|s_j - 0.5|$ and box area A_j ; e^{β_1} is then the odds ratio per doubling of the local deviation. These analyses use the INT8 engines with FP32 fallback. What box shifts mean for monocular distance estimation follows from a flat-ground camera model (Additional file 1, Section A.4).

2.8 Latency, jitter and energy

Latency was measured on an otherwise idle GPU, one engine at a time, with CUDA events around each of 2000 executions after 100 warm-up runs, with and without CUDA graphs and with inputs already on the device. Pre-processing, host transfers and the Faster R-CNN detection heads are excluded, so the measurement isolates the compiled engine. We summarise latencies by the median (p50), the 99th percentile (p99), the maximum and the jitter $Q_{0.99} - Q_{0.5}$. Energy was read from the GPU energy counter $E(t)$ (NVML) during at least 20 s of sustained execution of n batches of size B , replayed as CUDA graphs as in the latency measurements, after 10 s of idling at power P_{idle} :

$$E_{\text{img}} = \frac{E(t_1) - E(t_0)}{nB}, \quad E_{\text{img}}^{\text{net}} = E_{\text{img}} - \frac{P_{\text{idle}}(t_1 - t_0)}{nB}. \quad (18)$$

Sustained runs average out the partial sampling of the built-in power sensor [55]. We logged clock and temperature and repeated every measurement three times, reporting the median. Memory traffic and the precision in which each layer is executed were read from the engine inspector (Eq. (A2)), and reference energies per operation [17] are listed in Table C1.

2.9 Robustness under adverse imaging conditions

Fog, snow, frost, brightness (glare), contrast, Gaussian sensor noise and motion blur follow the COCO-C protocol [39, 56] at severities 1–5 with fixed per-image seeds.

Low light, which COCO-C does not include, is simulated by gamma darkening with Poisson shot noise (Fig. C3). For every condition, severity and precision we report box mAP, head-input deviation and detection-error rate on the 500 analysis images. For the YOLOv10 detectors we also tried two label-free remedies: a robust calibration set in which half of the images carry a random condition, and keeping the input or the top-ranked convolution in FP16.

2.10 PEP-AI-guided diagnosis and repair

The main INT8 results use the default placement and entropy calibration of the toolchain. In addition, we re-quantized every model in a configuration that was chosen only with label-free criteria on the calibration split, namely the head-input SQNR and the output consistency with FP32 (Eq. (A6)). Three things could be changed: (i) the calibration method, (ii) the placement of operators whose measured amplification shows that they magnify upstream noise, such as the pre-activation batch normalisation of DenseNet, and (iii) selective precision. For selective precision, the sensitivity of a convolution is the median SQNR drop across it, $s_n = 20 \log_{10} a_n$ (for the input convolution, $\max(0, 40 \text{ dB} - \text{SQNR}_n)$). Either the k most sensitive convolutions are kept in FP16 at once (one-shot), or the network is re-measured after every exclusion (iterative; Algorithm 1). All variants share the calibration scales of the base model (Additional file 1, Section A.5). In contrast to Hessian-based allocation [26, 27] or reconstruction-based PTQ [22, 23], no gradients are needed. We compare the ranking with random subsets (three seeds), with the heuristic of keeping all depthwise convolutions in higher precision [24, 37] and with a ranking predicted from FP32 statistics alone. Task accuracy is measured only afterwards.

2.11 AI-based business optimisation (AIBO) scenario

We take the measured energy per image at batch size 6, which corresponds to the synchronised frames of the six cameras of a vehicle, and scale it to a fleet of $N = \text{vehicles} \times \text{cameras} \times \text{frame rate} \times \text{operating hours inferences per year}$. As reference values we use the EU non-household electricity price of 0.1837 EUR/kWh (second half of 2025) [57] and the EU grid intensity of 242 g CO₂/kWh (2023) [58], bracketed by 0.10–0.30 EUR/kWh and 100–400 g/kWh. Following the AIBO view of AI as an input to managerial decisions [12], the annual cost of a precision p is

$$C(p, \lambda) = N E_{\text{img}}(p) c_{\text{el}} + N K(p) \lambda, \quad (19)$$

with electricity price c_{el} , penalty λ per critical error and $K(p)$ critical errors per image relative to FP32. For the detectors, a critical error is an FP32 detection of a road user (person, bicycle, car, motorcycle, bus, truck) that vanishes in the deployed engine under the score-tolerant definition; for the classifiers, it is a top-1 disagreement. Because $K(\text{FP32}) = 0$, the *break-even penalty*

$$\lambda^*(p) = \frac{(E_{\text{img}}(\text{FP32}) - E_{\text{img}}(p)) c_{\text{el}}}{K(p)} \quad (20)$$

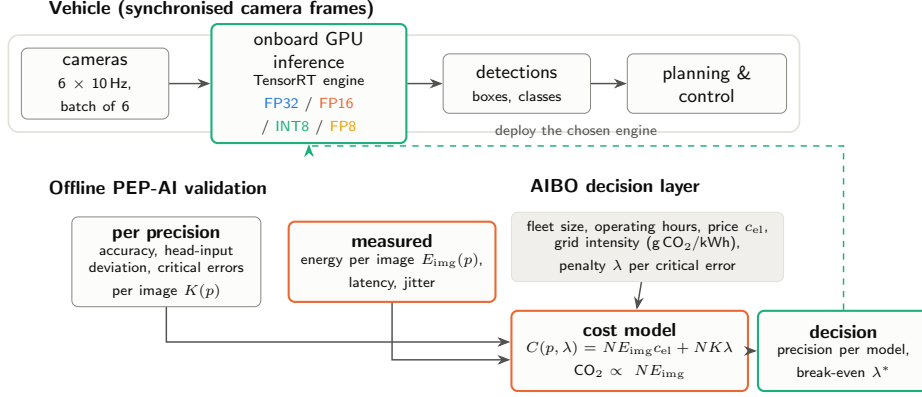


Fig. 3 AIBO view of quantized perception. Offline PEP-AI validation supplies accuracy, deviation, the critical errors per image $K(p)$ and the measured energy per image $E_{\text{img}}(p)$ of every precision; combined with fleet parameters, electricity price, grid intensity and a penalty λ per critical error, the cost model of Eq. (19) selects the precision of each deployed engine.

is the cost per critical error above which the saving of p no longer pays off, and the lower envelope of $C(\cdot, \lambda)$ gives the cost-optimal precision for every λ (Fig. 3). We count only the GPU inference energy (no host, cooling or idle power), and we do not count false positives, such as duplicate boxes, as critical errors. The absolute figures describe a fleet whose onboard accelerator has the measured energy per image of the H200. For an embedded GPU they scale with that GPU’s own energy per image, while the relative savings and the structure of Eqs. (19)–(20) carry over.

2.12 Statistical analysis

Accuracy differences to FP32 carry 95% confidence intervals from 1 000 paired bootstrap resamples of the evaluation images (Eq. (A7)). The discriminative power of a deviation score is measured by the ROC AUC, i.e. the Mann–Whitney probability that an erroneous case scores higher than a correct one, with bootstrap intervals; odds ratios carry bootstrap intervals over detections. Where the spread across images matters, medians are reported with interquartile ranges. The network-level relations of Section 3.8 have only one point per network ($n = 5$). For them, rank agreement is the Spearman correlation with the exact one-sided permutation p -value over all $5!$ orderings; it is one-sided because the model fixes the direction (lower SQNR, higher loss). The relation between head-input SQNR and relative loss is an ordinary least-squares fit of $\log_{10}$ of the relative loss, with t -based 95% confidence intervals of the slope, and we assess its predictive accuracy by leave-one-out, predicting each network from a fit to the other four. All random choices are seeded.

2.13 Static analysis of the FP32 parameters

The first steps of the procedure in Section 3.8 need calibration images. A coarser version needs none at all: the weights and the batch-normalisation (BN) buffers of the FP32 model already determine most of the quantities involved, without an image and

without a forward pass (`43_static_analysis.py`). The analysis is static by design. Executing the model on an image folder (`44_validate_static.py`) serves only to check it (Section 3.9).

Weights. For every convolution and linear layer, with BN folded in as in deployment, the weights w_c of output channel c are quantized exactly: to INT8 with $s_c = \max |w_c|/127$ (one scale per output channel, as deployed) and, for comparison, with a single scale for the whole tensor, and to E4M3 with $s_c = \max |w_c|/448$. The SQNR is computed from the quantized and the original weights, without any approximation. By Lemma 1 it is about $52.9 \text{ dB} - 20 \log_{10} \kappa_w$ with $\kappa_w = \max |w_c|/\text{RMS}(w_c)$, so outlier weights lower it, whereas the number of weights does not matter.

Activations. A BN layer stores the running mean μ_c and variance v_c of its input, so its output channel c has mean β_c and standard deviation $\sigma_c = |\gamma_c| \sqrt{v_c/(v_c + \epsilon)}$. We model each channel as a Gaussian, $N(\beta_c, \sigma_c^2)$, passed through the activation function that follows. Which activation follows a BN, and whether its output enters a residual addition instead, is read from the symbolic graph of the model (`torch.fx`, which traces the computation without executing it); for models that cannot be traced, the order of the modules is used. A ReLU turns it into a rectified Gaussian with a point mass at zero, which uses only the positive half of the symmetric INT8 grid; SiLU leaves a small negative tail bounded by -0.28 . The tensor is the mixture of its channels. If the channels differ in width, this mixture is heavy-tailed: the per-tensor range α is then set by the widest channels, and the narrow ones are quantized coarsely. We choose the range that minimises the error power of Eq. (5) on this mixture. It is evaluated on 2048 synthetic samples per channel, drawn with a fixed seed, by a search over 60 logarithmically spaced candidates between 5% and 100% of the largest sample magnitude. This gives $\kappa = \alpha/\sigma$ and, by Lemma 1, the injected SQNR; for FP8 the maximum is used. The channel imbalance is reported as the ratio of the 90th to the 10th percentile of the channel RMS. Summing the injected noise of all N modelled tensors, and optionally of the weights, gives a static SQNR_{add} by Eq. (16), and the gain of every BN that cannot be folded follows from Eq. (A3). The analysis reports every SQNR also as a relative error $r = 10^{-\text{SQNR}/20}$ in percent of the signal, and it lists the tensors it cannot model (convolution outputs without a BN, other normalisation layers), whose noise is then missing from SQNR_{add} .

How the network enters. By the decomposition of Section 3.8, the static SQNR_{add} falls by 3 dB each time the number of quantized tensors doubles, that is, with depth. Width enters only through the channel imbalance, and spatial size and parameter count do not enter at all. The sum is dominated by the weakest tensors, those with a large κ from outlier channels, depthwise inputs, gates or attention blocks. There are also things the static analysis cannot see: the propagation factor itself, tensors without a BN (such as the outputs of residual additions), the heavier-than-Gaussian tails of real activations, and errors created by decoders and heads.

Example. Take MobileNetV2, which is not used elsewhere in this article. The BN after the projection convolution of block 6 (`features.6.conv.3`, no activation) has $C = 32$ channels with $\beta_c \approx 0$ and σ_c between 1.04 and 12.36, a tensor RMS of 3.60 and a channel imbalance of 3.8. The MSE-optimal range is $\alpha = 40.9$, i.e. $\kappa = 11.36$, and Eq. (6) gives $52.87 - 20 \log_{10} 11.36 = 52.87 - 21.11 = 31.8 \text{ dB}$; quantizing the

samples directly gives 31.2 dB (no clipping). The step $s = \alpha/127 = 0.32$ is only 0.03 of the widest channel but 0.31 of the narrowest. The narrowest channel therefore has an SQNR of only $10 \log_{10}(12 \sigma_c^2/s^2) = 10 \log_{10}(12/0.31^2) = 20.9$ dB, against 42.5 dB for the widest: the shared per-tensor scale makes the narrow channels about 22 dB noisier. Over the whole network, the mean injected noise power of the 52 modelled tensors corresponds to $-10 \log_{10}(\rho_n^2) = 39.1$ dB. This gives $\text{SQNR}_{\text{add}} = 39.1 - 10 \log_{10} 52 = 39.1 - 17.2 = 22.0$ dB without the weights and 20.3 dB with the 53 weight layers. FP8 gives 12.0 dB, because the E4M3 weights (about 32 dB) contribute as much noise as the activations.

3 Results

3.1 Deployment efficiency on the GPU

INT8 is the fastest deployment format for Faster R-CNN, YOLOv10-X and EfficientNet-B0 at both batch sizes and for YOLOv10-S at batch size 8. At batch size 1, YOLOv10-S runs just as fast in FP8 (within 1%), and DenseNet-121 is faster in FP8 (Fig. 4, Table 5). Compared with strict FP32, INT8 lowers the median latency 1.6–8.5-fold at batch size 1 and 2.3–11.8-fold at batch size 8 (Faster R-CNN: from 40.7 to 3.5 ms per batch). It is also 1.12–1.27 and 1.24–1.47 times faster than FP16, so its advantage does not rest on the FP32 baseline alone. At batch size 8, INT8 needs 72–91% less energy per image than FP32 and 37–49% less than FP16 (Faster R-CNN 281 instead of 3 184 mJ, YOLOv10-X 295 instead of 2 477 mJ; 80–91% less if only the energy above idle is counted), and its weights take up 25–32% of their FP32 size. With CUDA graphs, latency is highly deterministic. The jitter is at most 0.015 ms for every INT8 engine and 0.08 ms for any engine, and the slowest of 2 000 executions exceeds the median by at most 23% (EfficientNet-B0 in FP8, 0.64 versus 0.52 ms), which bounds the observed worst-case execution time. The three repeats of each measurement differ by a median of 0.1% in latency and 0.5% in energy (at most 8.8% and 6.0%), far less than the differences between formats. The engine inspector shows that the gains really come from the number format (Table C3): every quantized convolution runs on INT8 kernels, and INT8 cuts the memory traffic per inference to 14–26% of FP32 (FP16: 40–62%, FP8: 26–39%). FP8 is slower than INT8 wherever its activation functions run as separate kernels, which doubles the activation traffic of YOLOv10-X. It is faster for DenseNet-121 (1.23 versus 1.85 ms at batch size 8), because the conventional engines implement the concatenations of this network as hundreds of copies. Power-limit throttling, launch overhead and the cost of output conversion are discussed in Additional file 1, Section B.1.

3.2 Task accuracy of the quantized engines

The FP32 engines reproduce the published accuracies (Faster R-CNN 36.95 mAP versus 37.0, YOLOv10-S 46.0 versus 46.3, YOLOv10-X 54.0 versus 54.4). These values may look low to a reader outside object detection, but they are competitive: COCO mAP averages the precision over 80 classes and ten IoU thresholds up to 0.95 [46, 54], and current real-time detectors reach 51–55 [43, 59, 60]. For road users, the FP32

Table 5 Task accuracy, latency, throughput and energy per precision (TensorRT 11.3, NVIDIA H200)

Model	Precision	Accuracy (%)	Δ vs FP32 [95% CI]	p50 bs1 (ms)	p99 bs1 (ms)	Images/s bs8	mJ/image bs8
Faster R-CNN R50-FPN	FP32	37.0	-	6.29	6.32	197	3184
	FP16	36.9	-0.01 [-0.04, +0.02]	0.91	0.92	1571	443
	INT8	36.7	-0.25 [-0.36, -0.13]	0.74	0.74	2312	281
	FP8	36.6	-0.37 [-0.49, -0.25]	0.91	0.92	1770	392
YOLOv10-S	FP32	46.0	-	1.78	1.78	1269	409
	FP16	46.0	+0.01 [-0.03, +0.03]	0.81	0.81	4370	109
	INT8	43.8	-2.24 [-2.56, -1.83]	0.69	0.69	5430	63.3
	FP8	45.6	-0.41 [-0.53, -0.28]	0.68	0.69	4297	96.7
YOLOv10-X	FP32	54.0	-	6.06	6.07	239	2477
	FP16	54.0	-0.01 [-0.07, +0.02]	1.99	2.00	1160	582
	INT8	47.8	-6.21 [-6.52, -5.74]	1.57	1.57	1658	295
	FP8	53.9	-0.15 [-0.31, -0.01]	1.79	1.79	1375	430
EfficientNet-B0	FP32	65.8	-	0.59	0.60	7086	45.7
	FP16	65.8	-0.02 [-0.12, +0.06]	0.42	0.42	12087	20.6
	INT8	24.8	-40.97 [-42.07, -39.77]	0.38	0.38	16564	12.7
	FP8	65.4	-0.39 [-0.79, 0.00]	0.52	0.52	9598	26.0
DenseNet-121	FP32	62.0	-	2.78	2.79	1912	152
	FP16	62.0	-0.02 [-0.11, +0.07]	1.59	1.59	3344	70.5
	INT8	57.4	-4.68 [-5.37, -3.98]	1.39	1.39	4325	40.6
	FP8	61.7	-0.32 [-0.64, +0.02]	0.71	0.71	6510	40.9

Accuracy: COCO val2017 box mAP@[.5:.95] (5000 images) for detectors, ImageNetV2 top-1 (9000 images) for classifiers; 95% CI of the difference from 1000 paired bootstrap resamples of the images. Latency: median (p50) and 99th percentile (p99) of 2000 executions with CUDA graphs; throughput from the mean latency with CUDA graphs. Energy: gross GPU energy per image from the NVML counter during sustained CUDA-graph execution. Median of three repeats. INT8 and FP8 engines keep non-quantized operations in FP16; INT8 uses the entropy calibration of the toolchain on the full calibration set in its stored order (sensitivity: Table C16); Faster R-CNN: backbone and FPN only.



Fig. 4 Deployment efficiency on the NVIDIA H200 (TensorRT 11.3, CUDA graphs). **a**, **b** Speed-up of the median latency over strict FP32 at batch sizes 1 and 8. **c** Deployed weight size. **d** Gross GPU energy per image at batch size 8 (NVML, sustained CUDA-graph execution; logarithmic axis).

detectors reach an AP_{50} of 82–86% for persons and 68–76% for cars, and they find 88–93% of the large ones at the operating threshold (Table C5). The ImageNetV2 accuracies of the classifiers, 65.8% and 62.0%, correspond to 77.7% and 74.4% on the original ImageNet validation set [61]; the gap is the known shift of ImageNetV2 [47, 62]. FP16 matches FP32 within 0.02 points, with paired 95% confidence intervals that include zero (Table 5). Every INT8 loss, by contrast, is significant, and its size depends strongly on the architecture. Faster R-CNN loses 0.25 mAP (CI 0.13–0.36), YOLOv10-S and YOLOv10-X lose 2.2 and 6.2 mAP, DenseNet-121 loses 4.7 top-1 points, and EfficientNet-B0 collapses from 65.8% to 24.8%. FP8 loses at most 0.4 points. This is consistent with Eq. (7), by which the relative precision of FP8 does not depend on the range. Only for Faster R-CNN, whose INT8 quantizers operate at moderate ranges ($\kappa \approx 18$), is FP8 marginally worse than INT8 (36.58 versus 36.71). The head-input SQNR follows the accuracy: 15.0, 13.4 and 16.7 dB for FP8 against 11.6, -0.9 and 6.6 dB for INT8 in YOLOv10-S, EfficientNet-B0 and DenseNet-121.

The YOLOv10 losses concentrate on large objects ($mAP_{large} -7.7$ and -13.2), even though their localisation is unchanged (best IoU 0.947 versus 0.950 for YOLOv10-X). For YOLOv10-X the cause turned out to be structural. Its NMS-free one-to-one head suppresses duplicates implicitly [43], and INT8 breaks this suppression: 15.2% of its

detections above the operating threshold duplicate a higher-scoring box of the same class ($\text{IoU} \geq 0.7$), against 0.25% for FP32 and FP8 (Table C6). Adding class-wise non-maximum suppression, which leaves FP32 and FP8 unchanged, raises the INT8 mAP from 47.8 to 52.4 and $\text{mAP}_{\text{large}}$ from 57.2 to 65.7, at a cost of 0.16 ms per image (10% of the engine latency at batch size 1). The rest of the loss coincides with lower class scores for large objects (median 0.92 in FP32, 0.85 in INT8). YOLOv10-S shows no such effect (0.55% duplicates). Its default quantizer placement, however, had cost it another 0.9 mAP through the quantized TopK and box decoder, a deployment artefact that output-level metrics would have blamed on the network itself (Additional file 1, Section A.1).

For the fragile networks, the INT8 accuracy also depends on how the ranges are set (Tables C17 and C16). The toolchain builds its entropy histogram one image at a time, so the first calibration image fixes a lower bound for every clipping range (Additional file 1, Section A.1). Calibration subsets that begin with a different image leave Faster R-CNN, the network with the smallest propagation factor, unchanged (± 0.01 mAP). They move YOLOv10-X by up to 2.7 mAP, however, and EfficientNet-B0 from 24.8% to 41.7%. An order-independent entropy calibration raises DenseNet-121 to 60.0% but over-clips the SiLU networks (YOLOv10-X 37.1 mAP, EfficientNet-B0 0.1%). Quantizing only the inputs of convolutions, the placement used by Wu et al. [18], lifts EfficientNet-B0 to 54.9% with the order-independent calibration but leaves it at 23.8% with the calibration of the toolchain. In short, no INT8 range setting is best for all networks. FP8, whose precision does not depend on the range (Lemma 1), stays within 0.4 points of FP32 for all of them (Additional file 1, Section B.2).

3.3 Activation-level deviation

The FP16 engines deviate from FP32 by a head-input SQNR of 36.9–50.5 dB, which is the numerical floor of the comparison. The INT8 deviations are one to two orders of magnitude larger, and they are ordered like the accuracy losses (Table 6): the head-input MRE_{proj} is 2.0% for Faster R-CNN, 5.8% for YOLOv10-S, 13.4% for DenseNet-121, 14.4% for YOLOv10-X and 25.9% for EfficientNet-B0. For every detector, the relative deviation is lower inside the ground-truth object regions than in the background (Faster R-CNN 2.9% vs. 3.3%, YOLOv10-S 3.9% vs. 6.2%, YOLOv10-X 6.4% vs. 13.2%; Fig. 5). This is what the additive model predicts: the same absolute noise is smaller relative to the stronger object responses (Eq. (A4)). The hexbin densities (Fig. 6) show why fragile networks fail. For EfficientNet-B0, a diagonal of deviations equal to minus the FP32 value marks activations that a too coarse per-tensor scale has rounded to zero.

Along the depth (Fig. 7), the SQNR falls within the first layers and then levels off, as Proposition 4 predicts. Fitting the chain recursion to consecutive convolutions gives $g = 0.90, 0.60$ and 0.70 for Faster R-CNN, YOLOv10-S and YOLOv10-X ($R^2 = 0.68, 0.13$ and 0.24 , because branches break the chain order). The corresponding fixed points, 18.5, 13.7 and 5.7 dB, are close to the measured plateaus of 20.3, 13.5 and 6.0 dB. The densely branched EfficientNet-B0 and DenseNet-121 do not form chains; for them the graph form of Proposition 3 is needed. FP8 lies between INT8 and FP16 along the whole depth (median layer SQNR 14.3–21.5 dB, against 5.0–19.8 dB

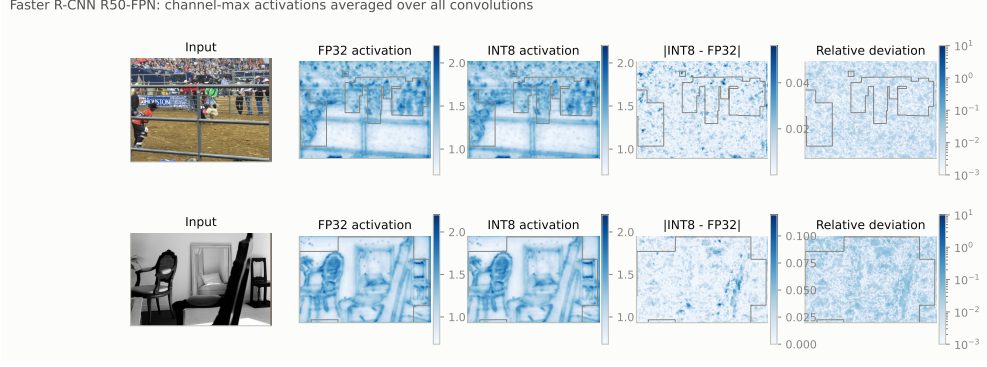


Fig. 5 FP32 and INT8 activation maps of Faster R-CNN. Channel-wise maximum activations of the Faster R-CNN backbone and FPN, averaged over all convolutions, for FP32 and INT8 on a shared colour scale, with the absolute and relative deviation. Grey contours: ground-truth boxes.

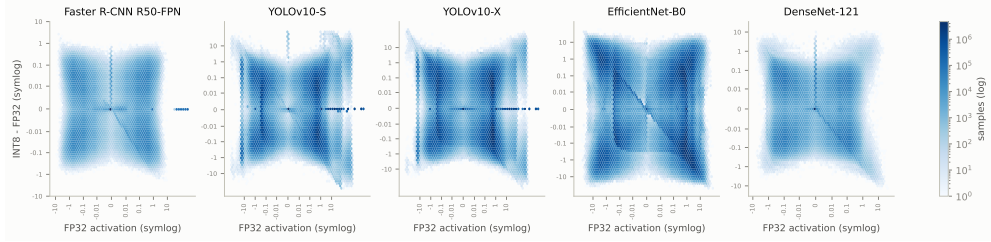


Fig. 6 Density of INT8 deviation versus FP32 activation. Symmetric-logarithmic axes (128 random elements per tensor and image, 500 images per model).

for INT8). The operator ratios (Fig. C4) quantify, per operator type, the attenuation described in the conference paper. Sigmoid gates attenuate ($a_n = 0.30\text{--}0.54$), max pooling attenuates only where no quantizer precedes it, and non-folded batch normalisation amplifies (median 1.17 per layer in DenseNet). The amplification is predicted by the closed form of Eq. (A3) from the FP32 parameters alone (median 1.24; Spearman $\rho = 0.83$ over 62 layers).

The additive model ties these observations to the fidelity of the network as a whole (Table C7, Fig. C7). On its own, every INT8 quantizer is benign: the median injected SQNR is 26.7–32.5 dB, and Eq. (6) predicts it within 1.1–2.8 dB. Summed with unit propagation factors, the 62–187 quantizers upstream of the head predict head-input SQNRs of 2.6–12.5 dB, without executing a quantized model. The effective propagation factor of Eq. (16) then ranks the networks almost exactly as their relative INT8 loss (Spearman $\rho = 0.9$ over $n = 5$ networks, exact one-sided $p = 0.04$): it is 0.13 for Faster R-CNN, 0.28 and 0.69 for YOLOv10-S and YOLOv10-X, 0.90 for DenseNet-121 and 2.89 for EfficientNet-B0. The injected noise is thus nearly the same across architectures; what decides the INT8 loss is $\bar{\Gamma}$.

Table 6 Activation-level deviation of INT8 from strict FP32 (500 images per model)

Model	Head input (INT8)			All convolutions (INT8)			Head SQNR FP16	
	MRE _{proj} (%)	MRE _{elem} (%)		SQNR (dB)	MRE _{elem} (%)	worst SQNR (dB)	(dB)	
Faster R-CNN R50-FPN	2.0 [1.7, 2.3]	7.1 [6.0, 8.3]	21.5 [20.1, 23.2]	8.6 [7.4, 10.1]	12.2	50.5		
YOLOv10-S	5.8 [5.1, 6.6]	16.9 [14.5, 20.6]	11.6 [10.1, 13.0]	17.1 [15.6, 18.9]	9.5	45.7		
YOLOv10-X	14.4 [12.8, 16.1]	37.8 [30.4, 45.9]	4.3 [3.3, 5.4]	37.2 [32.0, 44.0]	2.0	36.9		
EfficientNet-B0	25.9 [20.9, 34.8]	81.4 [72.5, 91.1]	-0.9 [-1.8, -0.2]	48.6 [46.4, 50.5]	-1.2	42.8		
DenseNet-121	13.4 [10.8, 16.9]	20.9 [14.9, 28.1]	6.6 [5.0, 7.9]	32.4 [26.9, 38.3]	5.3	45.2		

Head input: tensors consumed by the task head. Median over images, interquartile range in brackets. All convolutions: per image, the median element-wise relative error over every convolution output. Worst SQNR: lowest per-layer median. INT8 with FP32 fallback, so the deviation is caused by quantization alone; the FP16 column gives the numerical floor of the comparison.

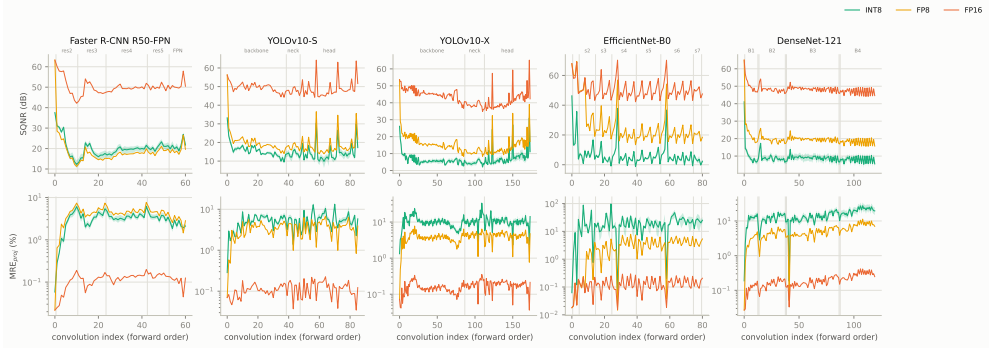


Fig. 7 Layer-wise propagation of quantization deviation. SQNR (top) and median relative error of the channel-max projection (bottom) of every convolution, ordered by depth, for INT8 (median over 500 images, band: interquartile range), FP8 (100 images) and FP16 (500 images; Faster R-CNN 100). Grey lines and labels: network stages.

3.4 Deviation and detection errors

At the operating threshold of 0.5, 35% of the validation images contain a strict FP32/INT8 disagreement for Faster R-CNN and YOLOv10-S (27.7% for YOLOv10-X). Most of this is threshold flicker, though. If INT8 counterparts are accepted down to a score of 0.3, only 6–11% of the images remain, and only 0.6%, 2.9% and 1.3% of the FP32 detections genuinely vanish. At the image level, the head-input deviation does not predict these errors (AUC 0.52, 0.40 and 0.39). A trivial covariate, the number of FP32 detections, does much better (0.80, 0.71 and 0.77), and the deviation level itself depends on the architecture. The link does hold locally, however (Fig. C10). Measured in the box of the FP32 detection, that is, where the reference network located the object, the deviation separates vanishing detections with a cross-validated AUC of 0.60–0.67. Their share rises 2.7-, 11.8- and 8.9-fold from the lowest to the highest decile, and each doubling of the local deviation multiplies the odds of vanishing by 1.70 (95% CI 1.36–2.07), 3.22 (2.59–4.01) and 4.25 (3.13–5.95) after adjustment for score margin and box size (Eq. (17)). The data-derived thresholds of the local deviation are 2.1%, 3.8% and 9.8% for Faster R-CNN, YOLOv10-S and YOLOv10-X, and detections above them vanish 2.0, 2.4 and 2.9 times as often as those below (0.94 vs. 0.47%, 4.2 vs. 1.8% and 2.5 vs. 0.85%). The deviation is therefore informative for individual detections, but not as an image-level threshold. Detections that survive are displaced only slightly. Even so, the 95th-percentile distance error of a monocular pedestrian estimate is 5.2% for Faster R-CNN, against 1.2–1.8% for YOLOv10 (Table C4, Additional file 1, Section B.4).

3.5 Robustness under adverse conditions

The INT8 Faster R-CNN keeps up with FP32 under all eight conditions (Fig. C11). Over 40 condition–severity pairs its mAP differs by between -1.0 and $+0.3$ points, while the corruptions themselves cost FP32 up to 34 points. Its internal deviation does grow under conditions that lower the image contrast (head-input SQNR from 21.7 to

14.5 dB; Fig. C13). For the YOLOv10 detectors, this growth shows up in the accuracy. On the 500 analysis images, the relative INT8 loss grows from 2.9% and 8.1% on clean images to 14.8% and 36.2% at the strongest contrast reduction (YOLOv10-X: 25.8 instead of 40.4 mAP). At the same time the head-input deviation, which needs no labels, roughly doubles, so it could signal the penalty in a shadow comparison with the FP32 model. FP8, with the same static scales, stays within 0.4–1.9% of FP32 for YOLOv10-X at every contrast severity (Fig. C11), as Corollary 2 predicts.

Looking at individual quantizers, the INT8 penalty follows the $20 \log_{10} c$ of Corollary 2 once the attenuation c is measured at the activations. In YOLOv10-X, clipping in the self-attention block adds another 11–15 dB, whereas no FP8 quantizer loses more than 0.5 dB (Fig. C12). None of the label-free INT8 remedies removed the penalty (robust calibration recovered at most 0.3 mAP, an FP16 input or top-ranked convolution at most 1.2 mAP). Max calibration even doubled it, because its coarser steps cost more than the clipping they avoid (Additional file 1, Section B.5).

3.6 From diagnosis to repair

For DenseNet-121, the label-free analysis points to a placement error. Keeping its pre-activation batch normalisation in FP16 raises the INT8 top-1 accuracy from 57.4% to 61.6% (FP32: 62.0%) and the head-input SQNR on the calibration split from 6.6 to 17.5 dB, and the label-free criteria do select this variant (Table C2). The calibration method of the YOLOv10 detectors, on the other hand, cannot be chosen without labels (Additional file 1, Section B.6).

Selective precision repairs damage that is concentrated in a few layers (Fig. C14, Table C15). In EfficientNet-B0, keeping just the first depthwise convolution in FP16 raises the top-1 accuracy from 24.8% to 43.9%. The iterative ranking reaches 62.3% with 8 and 66.0% with 20 of the 82 convolutions in FP16, against 45.8% and 57.2% for the one-shot ranking and 34.5–45.7% for random subsets of 20. The architectural heuristic from the literature, all 16 depthwise convolutions in FP16 [24, 37], reaches 65.0%, as much as the iterative ranking with 17 convolutions. It does so, however, at the highest latency and energy of all variants (0.95 ms and 31.0 mJ per image at batch size 8, against 0.72 ms and 23.3 mJ for the iterative ranking at 62.3%). The large gap between one-shot and iterative ranking reflects how far this network departs from the additive model ($\bar{\Gamma} = 2.89$). Along the depth (Fig. 8), eight FP16 convolutions lower the deviation in the first quarter of the network from 12.8% to 0.9%, but every variant above 54% top-1 is slower than pure FP16. YOLOv10-X recovers 5.1 of its 6.2 lost points with 20 ranked convolutions and remains faster than FP16. The convolutions ranked 13 to 20 are the ones that restore its duplicate suppression (15.8% duplicates with 12 and 0.4% with 20 FP16 convolutions; Table C6). YOLOv10-S, which spreads its noise over 79 quantizers of similar strength ($\bar{\Gamma} = 0.28$), gains nothing (43.8–44.0 mAP). A ranking predicted from FP32 statistics alone does not work (EfficientNet-B0 reaches 34.9% with 20 layers): the injected noise is predictable, but its propagation $\Gamma_{n \rightarrow h}$ has to be measured.

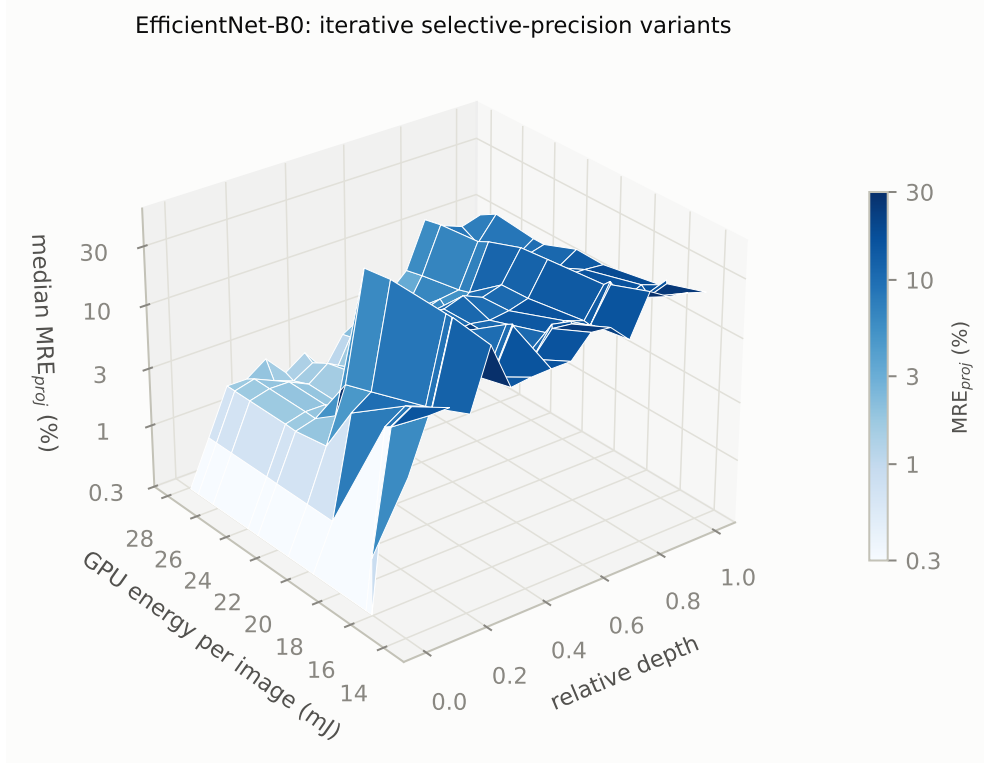


Fig. 8 Deviation, depth and energy of selective-precision variants of EfficientNet-B0. Median MRE_{proj} of every convolution against its relative depth (16 bins) and the GPU energy per image of the variant, for the fully quantized engine and the iterative ranking with $k = 1\text{--}20$ convolutions in FP16 (64 images each; batch size 8, shorter protocol without CUDA graphs, comparable only between variants). Logarithmic vertical axis and colour scale; top-1 accuracy in Table C15.

3.7 Fleet-level energy, cost and emissions

Our example fleet has 1000 vehicles with six cameras at 10 Hz, which amounts to 1.26×10^{12} images per year. At the fleet batch size of 6, INT8 needs 70–91% less energy per image than FP32 and 36–50% less than FP16. With strict FP32, the Faster R-CNN backbone would consume 1 136 MWh per year and YOLOv10-X 885 MWh. INT8 brings this down to 99 and 104 MWh, which at the reference price and grid intensity saves 191 kEUR and 251 t CO_2 per year for Faster R-CNN and 144 kEUR and 189 t for YOLOv10-X (Faster R-CNN: 104–311 kEUR and 104–415 t over the price and grid ranges; Fig. 9, Table 7). FP16 already captures most of this saving; INT8 adds another 62 and 105 MWh per year on top of it. Counting only the energy above idle lowers all savings by about a fifth (Additional file 1, Section B.7).

The cost model of Eq. (19) turns these savings into a decision. Relative to FP32, the deployed INT8 engines lose 0.010–0.038 road-user detections per image (FP8 0.003–0.044, FP16 at most 0.002), which means $1\text{--}5 \times 10^{10}$ single-frame critical errors per

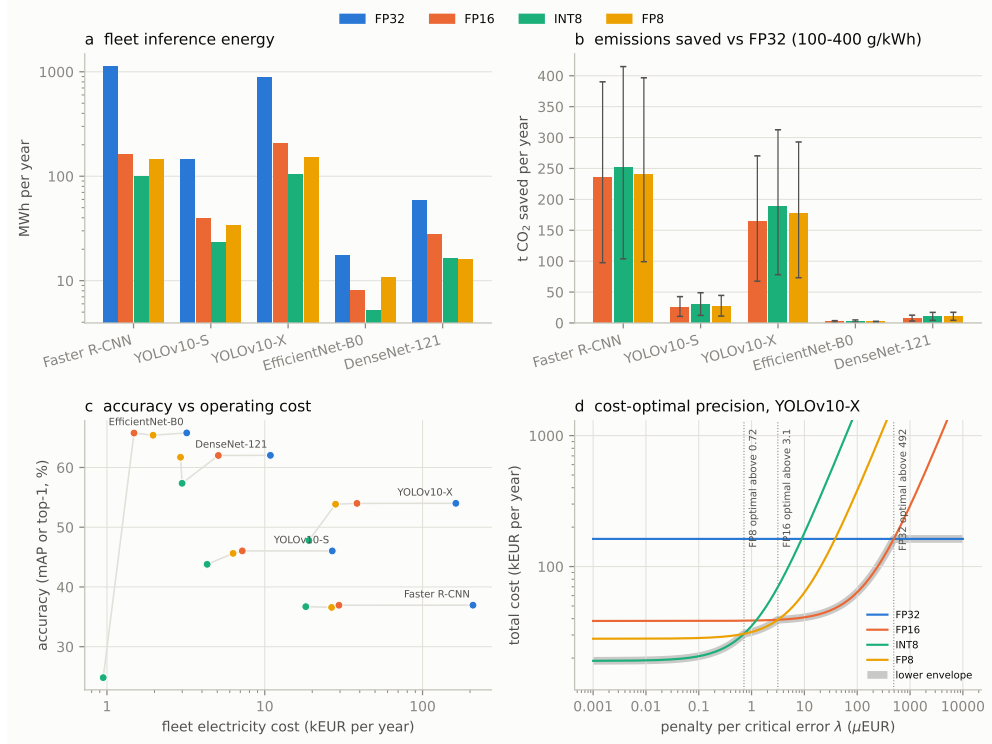


Fig. 9 AIBO fleet scenario. 1 000 vehicles, six cameras at 10 Hz, 16 h/day, measured energy at batch size 6, i.e. the synchronised frames of the six cameras. **a** Annual inference energy per precision. **b** CO₂ saved relative to FP32 (bars: 242 g/kWh; whiskers: 100–400 g/kWh). **c** Task accuracy against annual electricity cost. **d** Total annual cost of Eq. (19) for YOLOv10-X as a function of the penalty per critical error; the grey lower envelope marks the cost-optimal precision.

year for the INT8 fleet. The lower envelope of the cost lines (Fig. 9d) gives the cost-optimal precision. For YOLOv10-X, INT8 is optimal only for $\lambda < 0.72 \mu\text{EUR}$, FP8 up to $3.1 \mu\text{EUR}$, FP16 up to $492 \mu\text{EUR}$, and FP32 above that. For Faster R-CNN and YOLOv10-S, INT8 is optimal below $0.24 \mu\text{EUR}$ and FP16 above. These thresholds are small because the saving is spread over more than 10^{12} frames. In a real perception stack, on the other hand, tracking compensates for most single-frame misses near the score threshold. If such misses are practically free, INT8 captures the full saving; if they carry any measurable cost, FP8 or FP16 keeps most of it (94% and 87% of the INT8 saving for YOLOv10-X) at a fraction of the errors.

3.8 Predicting INT8 tolerance from the noise and propagation model

The analysis so far can be condensed into a procedure that is applied to a new network before it is deployed. Table C8 lists its formulas for INT8, FP8 and FP16, Table C10 applies it to all five networks, and Table C9 walks through every step for YOLOv10-S

Table 7 AIBO fleet scenario: annual inference energy, cost, emissions and break-even error penalty per precision

Model	Precision	Energy (MWh/yr)	Cost (kEUR/yr)	CO ₂ (t/yr)	Saved vs FP32 (kEUR/yr)	(t CO ₂ /yr)	Critical errors (per 10 ³ images)	Break-even (µEUR/error)
Faster R-CNN R50-FPN	FP32	1136	209	275	—	—	—	—
	FP16	161	30	39	179	236	1.6	88.8
	INT8	99	18	24	191	251	38.4	3.9
	FP8	144	27	35	182	240	44.4	3.3
YOLOv10-S	FP32	146	27	35	—	—	—	—
	FP16	39	7	9	20	26	0.0	—
	INT8	23	4	6	22	30	9.6	1.9
	FP8	34	6	8	20	27	4.6	3.5
YOLOv10-X	FP32	885	163	214	—	—	—	—
	FP16	209	38	51	124	164	0.2	492
	INT8	104	19	25	144	189	12.8	8.9
	FP8	153	28	37	134	177	2.8	38.1
EfficientNet-B0	FP32	17	3	4	—	—	—	—
	FP16	8	1	2	2	2	3.4	0.4
	INT8	5	1	1	2	3	718.8	0.0025
	FP8	11	2	3	1	2	76.7	0.013
DenseNet-121	FP32	59	11	14	—	—	—	—
	FP16	28	5	7	6	8	3.0	1.5
	INT8	16	3	4	8	10	239.3	0.026
	FP8	16	3	4	8	10	60.8	0.1

1 000 vehicles, 6 cameras at 10 Hz, 16 h/day; measured gross GPU energy per image at batch size 6 (NVIDIA H200), i.e. the synchronised frames of all cameras of a vehicle; electricity price 0.1837 EUR/kWh and grid intensity 242 g CO₂/kWh. Critical errors: FP32 road-user detections that vanish in the deployed engine (score-tolerant definition); classifiers: top-1 disagreements with FP32. Break-even: penalty per critical error at which the precision costs as much as FP32.

and YOLOv10-X; Fig. C6 relates the predictions to the measured accuracy. A short reminder on reading SQNR differences: 6 dB is a factor of two in the amplitude of the relative error, 20 dB a factor of ten, and a head-input SQNR of 0 dB means that the deviation is as large as the signal.

Step 1: noise of each quantizer, from FP32 statistics. The calibrated scales and the FP32 activations give the normalised range κ of every quantizer and, through Lemma 1, its injected noise. Over the 62–187 quantizers upstream of the head of each network, the median of Eq. (6) lies within 0.6–2.9 dB of the exact value (mean absolute error 1.1–2.8 dB). For the 79 quantizers of YOLOv10-S, for example (median $\kappa = 15.3$), it predicts 29.4 dB against an exact 28.0 dB. The FP8 prediction of Eq. (7), 31.5 dB for every quantizer, matches the exact values of all five networks within 0.02 dB. A single INT8 quantizer is therefore benign and predictable (26.7–32.5 dB). Where the networks differ is in how many quantizers they contain and in how the noise propagates.

Step 2: network-level screening without a quantized model. Summing the injected noise with unit propagation factors (Eq. (16)) predicts a head-input SQNR of 2.6–12.5 dB for INT8 and 10.1–14.6 dB for FP8. This prediction uses FP32 statistics only, and it already orders the networks by their relative INT8 loss with a single exchange (Spearman $\rho = 0.8$, $n = 5$, exact one-sided $p = 0.07$; Fig. C6a). Faster R-CNN, with the highest value, loses 0.7% of its accuracy, whereas YOLOv10-X and EfficientNet-B0, with the lowest, lose 11.5% and 62.3%. A log-linear fit gives a tenfold loss per 6.2 dB ($R^2 = 0.77$). Predicting each network from the other four, however, misses by up to a factor of 5.9 (EfficientNet-B0). This step should therefore be read as a screening that separates safe from suspicious networks, not as a predictor of the loss. For FP8 the same computation gives at least 10 dB for every network, and indeed every FP8 engine stays within 1.0% of its FP32 accuracy.

Step 3: one measurement of the quantized head input. Once the head-input SQNR of the quantized network is measured, it orders the networks exactly by their relative INT8 loss ($\rho = 1.0$, exact one-sided $p = 0.008$; Fig. C6b), and the loss grows tenfold per 12.2 dB of lost SQNR ($R^2 = 0.96$). Predicted from the other four networks, the relative loss of each network is met within a factor of 1.3–2.2 (Table C10). The largest miss, EfficientNet-B0 with 28.7 instead of 62.3%, lies at the end of the range, where the network collapses. The difference between this measurement and the FP32-only prediction is the effective propagation factor, $\text{SQNR}_h = \text{SQNR}_{\text{add}} - 10 \log_{10} \bar{\Gamma}$. Faster R-CNN gains 8.9 dB by attenuating the injected noise ($\bar{\Gamma} = 0.13$), YOLOv10-S gains 5.6 dB, YOLOv10-X 1.6 dB and DenseNet-121 0.5 dB, whereas EfficientNet-B0 loses 4.6 dB because it amplifies the noise ($\bar{\Gamma} = 2.89$). Note that EfficientNet-B0 does not inject more noise than YOLOv10-X (3.7 against 2.6 dB). Its collapse is a property of the propagation, and only the measurement reveals it.

Step 4: depth and deployment conditions. For the chain-like detectors, the fixed point of Eq. (15) predicts the plateau of the layer SQNR within 0.3 dB (13.7 against 13.5 dB for YOLOv10-S, 5.7 against 6.0 dB for YOLOv10-X; Table C9). When the input changes after calibration, Corollary 2, combined with the measured attenuation at each quantizer input, predicts the change of the injected SQNR at contrast severity 5 within 0.3 dB (−1.2 against −1.2 dB and −0.5 against −0.8 dB), and it correctly predicts that FP8 shows no such change. The cumulative change of the layer SQNR is,

however, 2.9–4.5 dB larger, because the shifted input also moves the operating point of the network. Corollary 2 thus predicts the direction of the effect and the difference between INT8 and FP8, but not its full size.

Step 5: choice of format and of layers. A network that passes the screening of step 2 and reaches a high head-input SQNR in step 3, such as Faster R-CNN (21.5 dB, relative loss 0.7%), can be deployed in INT8. For a network with a low head-input SQNR, step 1 tells us that FP8 removes the dependence on the range. Where FP8 kernels are not available, Proposition 3 selects the layers to keep in FP16 by their measured contribution; the propagation factors of single layers cannot be predicted from FP32 statistics (Section 3.6). The procedure is implemented in `scripts/42.predict_int8.py` of the code repository.

Which parts of a network drive the error. By Eq. (16), $\text{SQNR}_{\text{add}} = -10 \log_{10} N - 10 \log_{10} \langle \rho_n^2 \rangle$ holds exactly. The head-input fidelity therefore loses 3 dB each time the number N of quantizers on the way to the head doubles (62 for Faster R-CNN against 187 for YOLOv10-X and DenseNet-121, a difference of 4.8 dB). The mean injected noise, in turn, is dominated by the weakest quantizers, not by the typical one. EfficientNet-B0 has the highest median quantizer SQNR of the five networks (32.5 dB), yet its mean noise corresponds to 24.2 dB, 8 dB lower, because the inputs of its depthwise convolutions carry channel ranges that differ by a factor of 61 under one per-tensor scale (Table C7). Which quantizers are weak follows from Lemma 1: a large normalised range κ , caused by outlier channels, gates or attention blocks, costs 20 dB per decade. The propagation factor then depends on the operators between the quantizers and the head. Sigmoid gates, max pooling after unquantized layers and the residual trunk of Faster R-CNN attenuate (Section 3.3), whereas non-folded batch normalisation amplifies, by a factor that its parameters predict (Eq. (A3)). Table C14 summarises these effects. Weights matter little for INT8. With one scale per output channel, their median SQNR is 37–43 dB in all seven networks analysed statically (Table C11), against 26–34 dB with a single scale per tensor. In FP8 they reach 32 dB, as much as the activations, and so they contribute equally. The size of a network does not predict its tolerance: Faster R-CNN (26.8 M parameters) is the most robust network and EfficientNet-B0 (5.3 M) the most fragile one. Finally, decoders and heads can turn small deviations into structural errors that the head-input SQNR does not show, such as a quantized TopK or the duplicate suppression of the NMS-free YOLOv10 head (Section 3.2).

Mathematical status. It is worth being explicit about what is proved and what is not. Lemma 1, Corollary 2 and Propositions 3 and 4 are derived under the stated assumptions, with proofs in Additional file 1, Sections A.2 and A.3. Their constants were checked numerically: $10 \log_{10}(12 \cdot 127^2) = 52.87$ dB, $10 \log_{10}(8 \ln 2) = 7.44$ dB, and a Monte Carlo simulation of E4M3 rounding gives 31.53 dB against the predicted 31.52 dB. $\bar{\Gamma}$ is a definition, and the decomposition over N is an identity. The relation between head-input SQNR and relative loss, by contrast, is empirical. An ordinary least-squares fit of $\log_{10}$ of the relative loss on the head-input SQNR of the five networks gives a slope of -0.082 per dB (95% confidence interval -0.111 to -0.053 , $p = 0.003$), i.e. a relative loss proportional to $r_h^{1.64}$, with an exponent between 1.05 and 2.23. Two limiting cases bracket this exponent. Suppose a decision flips when the

perturbation of its score exceeds its margin, and the margins have a finite density at zero; then the share of flipped decisions grows linearly with the perturbation amplitude (exponent 1). For a metric that is smooth at the FP32 operating point, where its first derivative vanishes, the loss grows quadratically instead (exponent 2). The measured exponent lies between the two, which is consistent with a mixture of threshold flips and changes of the ranking that enters the average precision. We have not derived this interpretation, and the fit rests on five networks and two accuracy metrics.

3.9 Static analysis and its validation on unseen networks

The static analysis of Section 2.13 takes 20–35 s per network on a CPU and needs no image (Table C11). On the five networks of this article, its INT8 SQNR_{add} orders the measured relative INT8 loss with one exchange (Spearman $\rho = 0.9$, exact one-sided $p = 0.04$), just like the FP32-statistics screening of Section 3.8. Its absolute values, however, are 8–12.5 dB higher than those obtained with the calibration of the toolchain (20.6 against 12.5 dB for Faster R-CNN, 14.9 against 2.6 dB for YOLOv10-X). The toolchain quantizes more tensors, sets its ranges by entropy calibration and meets heavier tails than the Gaussian model assumes. We also applied the analysis to two networks that are not used elsewhere in this article, MobileNetV2 and ResNet-50, and to the two classifiers of the article. To validate it, we executed these networks on ImageNetV2 with simulated quantization on the CPU, placed at exactly the modelled tensors (Table C12; `44.validate_static.py`). Each tensor has one symmetric per-tensor scale, set on 128 calibration images to the MSE-optimal range for INT8 and to the maximum for FP8; weights are quantized per output channel, and 2000 disjoint images are used for evaluation. The validation run is only a check, and the analysis itself executes nothing. Since placement and calibration differ from the TensorRT toolchain, the check tests the method, not the deployment numbers of a particular runtime.

How accurate the static prediction is depends on the number format, as Lemma 1 leads one to expect. For FP8, the injected noise of every quantizer is predicted within 0.02–0.14 dB on all four networks (1–2% of the error amplitude), and the static SQNR_{add} equals the measured one within 0.05 dB (12.0, 11.5, 11.1 and 7.8 dB). The relative precision of a floating-point format does not depend on the distribution, so the Gaussian approximation simply does not matter here. For INT8, the prediction depends on the range and thus on the tails. It is accurate for MobileNetV2, whose quantized tensors are either bounded (ReLU6) or unrectified projections (median +0.4 dB, mean absolute error 1.1 dB, rank correlation 0.98 over its 52 tensors). For EfficientNet-B0, ResNet-50 and DenseNet-121, whose SiLU and unbounded ReLU outputs have heavier tails than the Gaussian model, it is 4–6 dB too optimistic (mean absolute error 5.0–6.3 dB, i.e. the error amplitude is underestimated by a factor of 1.8–2.1), and the ranking of single tensors is weak there (0.34–0.79). Even so, the static SQNR_{add} still orders the four networks by their simulated INT8 loss with one exchange.

Neither format allows the accuracy to be predicted statically, because the propagation factor is not visible in the parameters. The same FP8 noise of about 11 dB becomes 6.8 dB at the head of MobileNetV2 ($\bar{\Gamma} = 3.3$, loss 10.3%) but 12.0 dB at the

head of ResNet-50 ($\bar{\Gamma} = 0.9$, loss 2.8%), and 7.8 dB becomes 17.0 dB at the head of DenseNet-121 ($\bar{\Gamma} = 0.12$, loss 0.6%). One measurement of the head input closes this gap. The relation of Section 3.8, calibrated on the TensorRT engines of the five other networks, predicts the relative loss of the simulated engines within the 95% bootstrap interval in six of the eight cases (ResNet-50: 1.5 against 1.1% for INT8 and 3.7 against 2.8% for FP8; DenseNet-121: 2.5 against 2.5% and 1.5 against 0.6%; EfficientNet-B0 FP8: 8.6 against 7.1%; MobileNetV2 FP8: 9.8 against 10.3%). The two misses are EfficientNet-B0 in INT8 (29.7 against 36.8%, a factor of 1.24) and MobileNetV2 in INT8 (1.4 against 0.2%, both below 2%). Table C13 gives the same quantities as relative errors in percent of the signal. For ResNet-50 in FP8, for example, the static analysis predicts an injected error of 2.7% per quantizer and 26.7% for the whole network, exactly as measured; after propagation the head input deviates by 25.1%, and the top-1 accuracy drops by 2.8% relative to FP32. In INT8 the static analysis predicts 1.4% per quantizer and 16.2% for the network, whereas 2.6% and 31.2% are measured; the network attenuates this to 14.3% at the head input (relative loss 1.1%). None of these deviations needs labels: only the accuracy loss does. As a further test outside torchvision, we applied both scripts to a custom 1.2 M-parameter network with a SiLU stem, depthwise blocks with squeeze-and-excitation and Hardswish, ReLU residual bottlenecks, one convolution without BN and a GroupNorm layer, whose BN statistics were collected on 256 ImageNetV2 images (untrained weights, so that only the noise can be compared; code in the repository). The FP8 noise per quantizer was predicted within 0.01 dB (median) and the network value within 0.02 dB (16.60 against 16.58 dB), whereas the INT8 noise was again too optimistic, by 6.5 dB, as for the other networks with SiLU and ReLU outputs; the coverage report listed the convolutions without BN and the GroupNorm layer.

The static analysis is therefore useful for decisions that do not depend on the propagation. It tells whether weights need per-channel scales (ResNet-50: 15.8 against 7.4 dB with one scale per tensor), how much noise FP8 injects into a given network, which batch normalisations amplify noise, which structures (depthwise inputs, imbalanced channels, gates, attention) deserve suspicion, and how a network ranks among measured references. Its INT8 noise estimate is reliable to about 1 dB for networks with bounded or unrectified activations and to about 5–6 dB otherwise. It does not give a fitted accuracy number, and for a good reason. A curve fitted from the static SQNR_{add} to the measured INT8 loss of the five networks misses by up to a factor of ten in leave-one-out (YOLOv10-X: 114% predicted against 11.5% measured), because the propagation factor is not visible in the parameters, and for FP8 the measured losses do not even follow the static noise (all five networks stayed within 1.0%). What the analysis does give is an honest range. For INT8, the two measured networks next to the new one in the static ranking bracket its loss; for ResNet-50 these are Faster R-CNN and YOLOv10-S, giving 0.7–4.9%, and the simulated measurement found 1.1%. For FP8, the measured networks lost 0.3–1.0% with TensorRT and 0.6–10.3% in the simulated check, the upper end belonging to a network that amplifies the noise (MobileNetV2, $\bar{\Gamma} = 3.3$); ResNet-50 lost 2.8%. In the same way, the deviation to be expected at the head input follows from the static SQNR_{add} and the range of propagation factors measured so far, $\text{SQNR}_h = \text{SQNR}_{\text{add}} - 10 \log_{10} \bar{\Gamma}$: for ResNet-50 in FP8, $\bar{\Gamma}$ between 0.12

and 3.3 gives a relative deviation of 9–48% of the signal, and 25.1% was measured; in INT8 the range is 6–42%, and 14.3% was measured. With one measurement of the quantized head input, which needs only unlabelled images, the relative loss can be predicted within a factor of 1.4 wherever it exceeds 1% (six of the eight cases), as long as the head-input SQNR lies within the range of the calibration networks (−1 to 22 dB).

4 Discussion

How do these results fit with the literature? The collapse of EfficientNet-B0 under per-tensor INT8 matches the known fragility of depthwise-separable networks [24, 37]. Lemma 1 and $\bar{\Gamma}$ add a quantitative reason: EfficientNet-B0 amplifies the injected noise instead of attenuating it. The accuracy advantage of FP8 confirms the analysis of Kuzmin et al. [29], while the speed advantage of INT8 for four of the five networks confirms the hardware argument of van Baalen et al. [30]; which format is preferable depends on the architecture and on the compiler. The dependence of INT8 accuracy on the range setting agrees with the finding that no calibration method is best for all networks [18] and with the known role of the calibration set [21]. Our results add the order of the calibration images as a further, hidden factor. The PEP-AI ranking can be seen as a gradient-free counterpart of Hessian-based mixed precision [26, 27]. The corruption sensitivity reported for quantized classifiers [38] shows up for the detectors only under contrast loss, which Corollary 2 explains. Finally, NMS-free detectors [43, 60] turn duplicate suppression into a learned function of small score differences, and INT8 can break it.

4.1 Technological limitations

All measurements were taken on a data-centre GPU. The relative effects follow from kernels and memory traffic, which embedded automotive GPUs share, but absolute latencies and energies have to be re-measured on the target hardware. TensorRT supports only static, symmetric, per-tensor activation scales, and this underlies both INT8 failure modes we observed: the collapse of EfficientNet-B0, whose depthwise layers see channel ranges that differ by a factor of 61 under one per-tensor scale, and the low-contrast penalty of Corollary 2. Per-channel or dynamic activation scales would behave differently, and the INT8 accuracies of the fragile networks are specific to the range-setting procedure (Table C17). The analytical models assume additive noise and first-order linearisation. Both fail for values below half a quantization step and under saturation or clipping, so Proposition 3 is a guide rather than a guarantee; this is why we also evaluated the measurement-based iterative variant. The rank correlation between $\bar{\Gamma}$ and the INT8 loss and the relation between head-input SQNR and relative loss rest on five networks and two accuracy metrics. The out-of-sample check covers four classifiers in simulated quantization only and should be extended to further architectures, tasks and runtimes. The static analysis models only batch-normalised tensors, as Gaussian channels. It therefore misses residual sums, tensors without batch normalisation and heavy activation tails, and it cannot see the propagation factor. We did not evaluate post-training methods that adapt the weights on calibration data [20, 22], nor cross-layer equalisation [24], which addresses the channel imbalance

of depthwise layers directly and would likely reduce the INT8 loss of EfficientNet-B0; our analysis targets the default toolchain. COCO contains traffic scenes but is not a driving dataset, the adverse conditions are synthetic, and for Faster R-CNN only the backbone and FPN were quantized. Finally, a single image-level deviation threshold cannot separate threshold flicker from genuine misses, so release criteria have to be applied locally, at the level of individual detections.

4.2 Practical applicability

PEP-AI quantifies where a quantized model deviates, whether the deviation predicts detection errors and how it changes under adverse conditions, each with an uncertainty, which is what the safety argument for a perception component requires [34, 35]. Even before any engine is built, the static analysis estimates the noise a network will receive (Sections 2.13 and 3.9), and the procedure of Section 3.8 then tells which networks need attention and how much accuracy INT8 will cost them. From the measurements we can give a recommendation for each architecture (Table C18). Faster R-CNN can be deployed in INT8 as it is. YOLOv10-X needs its duplicate suppression restored, either by a class-wise NMS for 0.16 ms per image or by selective precision. For YOLOv10-S, whose loss is spread over the whole network, only a different range setting or FP8 helps. DenseNet-121 recovers its accuracy with batch normalisation in FP16, but at nearly the cost of FP16, and EfficientNet-B0 is best served by FP16. For networks with a large propagation factor, INT8 accuracy should be checked over several calibration orders and procedures and confirmed on labelled data. Where FP8 tensor cores are available, FP8 is the most robust choice for the networks that INT8 damages: it is nearly lossless, insensitive to the range setting and to contrast loss (Lemma 1), and, depending on kernel fusion, faster than FP16. At fleet scale this choice is worth up to 1 GWh, 191 kEUR and 251 t CO₂ per year for 1 000 vehicles, and the break-even analysis tells an operator how much residual risk per frame the saving can pay for [2, 10].

4.3 Future research directions

We see three natural extensions. The first concerns four-bit formats, which the evaluated GPU does not execute natively [63], and per-channel or dynamic activation scales. Lemma 1, Corollary 2 and $\bar{\Gamma}$ carry over to them and would predict, from FP32 statistics and a measured propagation factor, how far the precision can be reduced. The measured propagation factors also suggest a structural pattern that deserves a systematic test: in FP8, the network with dense concatenations attenuated the noise ($\bar{\Gamma} = 0.12$) and the residual network passed it on almost unchanged (0.88), whereas the two networks with depthwise, linear-bottleneck blocks amplified it (2.25 and 3.3). With four networks this is a hypothesis; measuring the propagation factor of a few dozen architectures, which needs only unlabelled images and a CPU, would show whether $\bar{\Gamma}$ can be predicted from the structure and thus close the gap between the static analysis and the accuracy. The second concerns embedded automotive platforms, recorded adverse-weather driving data [40] and temporal consistency across video frames, where threshold flicker becomes an observable safety property. The third is spatially adaptive

computation. The deviation is larger in the background, and for the Faster R-CNN backbone an ideal spatial early exit could skip at most about 31% of the computation (Additional file 1, Section B.3). This is an upper bound that motivates such an accelerator design, but does not yet demonstrate it.

5 Conclusion

The main message of this work is that the error of a post-training quantized network separates into two parts. The first part, the noise that each quantizer injects, follows from FP32 statistics. By Lemma 1 it is $52.9 \text{ dB} - 20 \log_{10} \kappa$ for INT8 and a constant 31.5 dB for FP8 E4M3, and the injected noise of a network adds up to SQNR_{add} , which loses 3 dB each time the number of quantized tensors doubles and is dominated by the weakest tensors. The second part, how this noise propagates to the task head, is a property of the network. It is captured by a single factor $\bar{\Gamma}$, which ranged from 0.13 (Faster R-CNN, attenuating) to 2.89 (EfficientNet-B0, amplifying) and can only be seen by measuring the quantized network once. It was this propagation factor, and not the size of the network, that decided the INT8 tolerance: the 26.8 M-parameter Faster R-CNN lost 0.25 mAP, while the 5.3 M-parameter EfficientNet-B0 lost 41 top-1 points. The analysis also separates what depends on the distribution of the activations from what does not. The noise of a floating-point format is independent of the distribution and of the calibrated range, so the FP8 noise of a network can be predicted from its parameters alone within 0.02–0.14 dB per quantizer, also for networks never seen before. The noise of INT8, on the other hand, depends on the range and therefore on the tails of the distribution. It is predicted within about 1 dB where the activations are bounded or unrectified, but 4–6 dB too optimistically for SiLU and unbounded ReLU outputs, and it degrades under contrast loss (Corollary 2) and under changes of the calibration sample that FP8 absorbs. The noise model, the propagation bounds and the selection rule are proved under stated assumptions and agree with the measurements to within a few decibels. The relation between head-input fidelity and accuracy, a tenfold relative loss per 12 dB, is empirical and rests on five networks; for networks outside the calibration it predicted the loss within a factor of 1.4 wherever the loss exceeded 1%.

In practice, these findings give a procedure for choosing a number format before deployment. A static analysis of the FP32 weights and batch-normalisation parameters takes less than a minute on a CPU and needs no data; it gives the FP8 noise, flags the structures that are critical for INT8 and ranks a network among measured references. One measurement of the quantized head input on unlabelled images then predicts the relative INT8 loss within a factor of about two. Where INT8 fails, measured layer sensitivities locate the damage, from a single depthwise convolution in EfficientNet-B0 to the layers that implement the duplicate suppression of YOLOv10-X. Where the damage is concentrated, keeping these layers in FP16 restores most of the accuracy (EfficientNet-B0 from 24.8% to 66.0%, YOLOv10-X 5.1 of 6.2 lost points). For single detections, the deviation inside the box predicts whether a detection vanishes (odds

ratio 1.7–4.2 per doubling), whereas the image-level deviation does not. INT8 is therefore safe for networks that attenuate the injected noise, and FP8 is the more robust choice for the others, provided their propagation factor is moderate.

The stakes of this choice are measurable. On an NVIDIA H200, INT8 was 1.6–11.8 times faster than FP32 and 1.1–1.5 times faster than FP16, and it needed 72–91% less energy per image than FP32. For a fleet of 1 000 vehicles this amounts to up to 1 GWh and 251 t CO₂ per year, and the cost model gives the precision that minimises energy cost plus the penalty for quantization-induced errors.

Acknowledgements. This research was carried out at the Doctoral School of Informatics of the University of Debrecen.

Declarations

- Funding: This research received no specific funding.
- Competing interests: The authors declare no competing interests.
- Ethics approval and consent to participate: Not applicable.
- Consent for publication: Not applicable.
- Availability of data and materials: COCO 2017 and ImageNetV2 are publicly available. The tables, figures and per-layer and per-detection results of the reported run are included in the code repository (directory `reference_results`).
- Materials availability: Not applicable.
- Code availability: The complete pipeline (quantization, deployment, measurement and analysis scripts, configuration and figure generation) is available at <https://github.com/EgyipTomi425/EchteAI>; the code of the conference paper is preserved under the tag `citds-2026`. An archived release will be deposited upon acceptance.
- Authors’ contributions: T.M.: conceptualisation, methodology, software, measurements, formal analysis, writing – original draft. A.H. and R.L.: conceptualisation, supervision, writing – review and editing. All authors read and approved the final manuscript.
- Use of AI tools: The authors used a large language model (Claude, Anthropic) as an assistant while revising the text of the manuscript and while developing parts of the analysis scripts. The research questions, the experimental design and all measurements are the authors’ own. Every reported number is computed by the published scripts from the measured data, and all derivations, tables and figures were checked by the authors and can be reproduced with the code repository. The authors take full responsibility for the content of this article.
- Prior publication: This article extends the conference paper [41]. New in this version are the GPU deployment with strict-FP32, FP16, INT8 and FP8 TensorRT engines for five architectures, the complete-benchmark accuracy with confidence intervals, the latency, jitter, memory-traffic and energy measurements, the analytical noise and propagation model with its experimental tests, the operator-level amplification estimates, the statistical deviation–risk analysis, the robustness study, label-free diagnosis and repair including selective precision, the prediction procedure for INT8 tolerance, the static analysis with its validation on further networks, and the AIBO

fleet scenario. Text reused from the conference paper is limited to the definition of the original metrics.

Additional files

Additional file 1 (PEP-AI-supplementary.pdf, PDF). *Supplementary methods, results and extended data.* Step-by-step proofs of Lemma 1, Corollary 2 and Propositions 3 and 4; implementation details of quantization, deployment, deviation–risk analysis and repair; supplementary results on deployment efficiency, calibration, activation deviation, detection errors, robustness, repair and the fleet scenario; extended Tables C1–C18 and Figures C1–C14.

References

- [1] Veres, M., Moussa, M.: Deep learning for intelligent transportation systems: A survey of emerging trends. *IEEE Transactions on Intelligent Transportation Systems* **21**(8), 3152–3168 (2020)
- [2] Sudhakar, S., Sze, V., Karaman, S.: Data centers on wheels: Emissions from computing onboard autonomous vehicles. *IEEE Micro* **43**(1), 29–39 (2023)
- [3] Strubell, E., Ganesh, A., McCallum, A.: Energy and policy considerations for deep learning in NLP. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 3645–3650 (2019)
- [4] Schwartz, R., Dodge, J., Smith, N.A., Etzioni, O.: Green AI. *Communications of the ACM* **63**(12), 54–63 (2020)
- [5] Wu, C.-J., Raghavendra, R., Gupta, U., Acun, B., Ardalani, N., Maeng, K., Chang, G., Behram, F.A., Huang, J., Bai, C., Gschwind, M., Gupta, A., Ott, M., Melnikov, A., Candido, S., Brooks, D., Chauhan, G., Lee, B., Lee, H.-H.S., Akyildiz, B., Balandat, M., Spisak, J., Jain, R., Rabbat, M., Hazelwood, K.: Sustainable AI: Environmental implications, challenges and opportunities. In: *Proceedings of Machine Learning and Systems (MLSys)*, vol. 4 (2022)
- [6] Lannelongue, L., Grealey, J., Inouye, M.: Green algorithms: Quantifying the carbon footprint of computation. *Advanced Science* **8**(12), 2100707 (2021)
- [7] Luccioni, S., Jernite, Y., Strubell, E.: Power hungry processing: Watts driving the cost of AI deployment? In: *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT)* (2024). <https://doi.org/10.1145/3630106.3658542>
- [8] Desislavov, R., Martínez-Plumed, F., Hernández-Orallo, J.: Trends in AI inference energy consumption: Beyond the performance-vs-parameter laws of deep learning. *Sustainable Computing: Informatics and Systems* **38**, 100857 (2023) <https://doi.org/10.1016/j.suscom.2023.100857>

- [9] Henderson, P., Hu, J., Romoff, J., Brunskill, E., Jurafsky, D., Pineau, J.: Towards the systematic reporting of the energy and carbon footprints of machine learning. *Journal of Machine Learning Research* **21**(248), 1–43 (2020)
- [10] European Commission: The European Green Deal. Communication from the Commission, COM(2019) 640 final, Brussels (2019)
- [11] European Commission: Sustainable and Smart Mobility Strategy – Putting European Transport on Track for the Future. Communication from the Commission, COM(2020) 789 final, Brussels (2020)
- [12] Duan, Y., Edwards, J.S., Dwivedi, Y.K.: Artificial intelligence for decision making in the era of Big Data – evolution, challenges and research agenda. *International Journal of Information Management* **48**, 63–71 (2019) <https://doi.org/10.1016/j.ijinfomgt.2019.01.021>
- [13] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., Kalenichenko, D.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713 (2018)
- [14] Krishnamoorthi, R.: Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342* (2018)
- [15] Nagel, M., Fournarakis, M., Amjad, R.A., Bondarenko, Y., van Baalen, M., Blankevoort, T.: A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295* (2021)
- [16] Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M.W., Keutzer, K.: A survey of quantization methods for efficient neural network inference. In: *Low-Power Computer Vision*, pp. 291–326. Chapman and Hall/CRC, Boca Raton (2022). <https://doi.org/10.1201/9781003162810-13>
- [17] Horowitz, M.: Computing’s energy problem (and what we can do about it). In: *IEEE International Solid-State Circuits Conference (ISSCC)*, pp. 10–14 (2014)
- [18] Wu, H., Judd, P., Zhang, X., Isaev, M., Micikevicius, P.: Integer quantization for deep learning inference: Principles and empirical evaluation. *arXiv preprint arXiv:2004.09602* (2020)
- [19] NVIDIA: TensorRT. <https://developer.nvidia.com/tensorrt>. Version 11.3 (2026)
- [20] Nagel, M., Amjad, R.A., van Baalen, M., Louizos, C., Blankevoort, T.: Up or down? Adaptive rounding for post-training quantization. In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pp. 7197–7206 (2020)

- [21] Hubara, I., Nahshan, Y., Hanani, Y., Banner, R., Soudry, D.: Accurate post training quantization with small calibration sets. In: Proceedings of the 38th International Conference on Machine Learning (ICML). PMLR, vol. 139, pp. 4466–4475 (2021)
- [22] Li, Y., Gong, R., Tan, X., Yang, Y., Hu, P., Zhang, Q., Yu, F., Wang, W., Gu, S.: BRECO: Pushing the limit of post-training quantization by block reconstruction. In: International Conference on Learning Representations (ICLR) (2021)
- [23] Wei, X., Gong, R., Li, Y., Liu, X., Yu, F.: QDrop: Randomly dropping quantization for extremely low-bit post-training quantization. In: International Conference on Learning Representations (ICLR) (2022)
- [24] Nagel, M., van Baalen, M., Blankevoort, T., Welling, M.: Data-free quantization through weight equalization and bias correction. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 1325–1334 (2019)
- [25] Zhao, R., Hu, Y., Dotzel, J., De Sa, C., Zhang, Z.: Improving neural network quantization without retraining using outlier channel splitting. In: Proceedings of the 36th International Conference on Machine Learning (ICML). PMLR, vol. 97, pp. 7543–7552 (2019)
- [26] Dong, Z., Yao, Z., Gholami, A., Mahoney, M.W., Keutzer, K.: HAWQ: Hessian Aware quantization of neural networks with mixed-precision. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 293–302 (2019)
- [27] Yao, Z., Dong, Z., Zheng, Z., Gholami, A., Yu, J., Tan, E., Wang, L., Huang, Q., Wang, Y., Mahoney, M.W., Keutzer, K.: HAWQ-V3: Dyadic neural network quantization. In: Proceedings of the 38th International Conference on Machine Learning (ICML). PMLR, vol. 139, pp. 11875–11886 (2021)
- [28] Banner, R., Nahshan, Y., Soudry, D.: Post training 4-bit quantization of convolutional networks for rapid-deployment. In: Advances in Neural Information Processing Systems, vol. 32 (2019)
- [29] Kuzmin, A., van Baalen, M., Ren, Y., Nagel, M., Peters, J., Blankevoort, T.: FP8 quantization: The power of the exponent. In: Advances in Neural Information Processing Systems, vol. 35 (2022)
- [30] van Baalen, M., Kuzmin, A., Nair, S.S., Ren, Y., Mahurin, E., Patel, C., Subramanian, S., Lee, S., Nagel, M., Soriaga, J., Blankevoort, T.: FP8 versus INT8 for Efficient Deep Learning Inference (2023). <https://doi.org/10.48550/arXiv.2303.17951>
- [31] Sakr, C., Kim, Y., Shanbhag, N.: Analytical guarantees on numerical precision

- of deep neural networks. In: Proceedings of the 34th International Conference on Machine Learning (ICML), pp. 3007–3016 (2017)
- [32] Lin, J., Gan, C., Han, S.: Defensive quantization: When efficiency meets robustness. In: International Conference on Learning Representations (ICLR) (2019)
 - [33] Hooker, S., Courville, A., Clark, G., Dauphin, Y., Frome, A.: What do compressed deep neural networks forget? arXiv preprint arXiv:1911.05248 (2019)
 - [34] International Organization for Standardization: ISO 21448:2022 Road vehicles – Safety of the intended functionality (2022)
 - [35] International Organization for Standardization: ISO/PAS 8800:2024 Road vehicles — Safety and artificial intelligence. Publicly Available Specification (2024)
 - [36] Mohseni, S., Wang, H., Xiao, C., Yu, Z., Wang, Z., Yadawa, J.: Taxonomy of machine learning safety: A survey and primer. *ACM Computing Surveys* **55**(8), 157 (2022) <https://doi.org/10.1145/3551385>
 - [37] Sheng, T., Feng, C., Zhuo, S., Zhang, X., Shen, L., Aleksic, M.: A quantization-friendly separable convolution for MobileNets. In: Workshop on Energy Efficient Machine Learning and Cognitive Computing for Embedded Applications (EMC2) (2018)
 - [38] Xiao, Y., Zhang, T., Liu, S., Qin, H.: Benchmarking the robustness of quantized models. arXiv preprint arXiv:2304.03968 (2023)
 - [39] Michaelis, C., Mitzkus, B., Geirhos, R., Rusak, E., Bringmann, O., Ecker, A.S., Bethge, M., Brendel, W.: Benchmarking robustness in object detection: Autonomous driving when winter is coming. arXiv preprint arXiv:1907.07484 (2019)
 - [40] Zhang, Y., Carballo, A., Yang, H., Takeda, K.: Perception and sensing for autonomous vehicles under adverse weather conditions: A survey. *ISPRS Journal of Photogrammetry and Remote Sensing* **196**, 146–177 (2023) <https://doi.org/10.1016/j.isprsjprs.2022.12.021>
 - [41] Menyhárt, T., Hajdu, A., Lakatos, R.: Quantization-induced error propagation: Activation analysis within the PEP-AI framework for sustainable high-efficiency and explainable AI. In: Proceedings of CITDS 2026 (2026)
 - [42] Ren, S., He, K., Girshick, R., Sun, J.: Faster R-CNN: Towards real-time object detection with region proposal networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **39**(6), 1137–1149 (2017)
 - [43] Wang, A., Chen, H., Liu, L., Chen, K., Lin, Z., Han, J., Ding, G.: YOLOv10: Real-time end-to-end object detection. In: Advances in Neural Information Processing

- [44] Tan, M., Le, Q.V.: EfficientNet: Rethinking model scaling for convolutional neural networks. In: Proceedings of the International Conference on Machine Learning (ICML), pp. 6105–6114 (2019)
- [45] Huang, G., Liu, Z., van der Maaten, L., Weinberger, K.Q.: Densely connected convolutional networks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2261–2269 (2017)
- [46] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L.: Microsoft COCO: Common objects in context. In: Proceedings of the European Conference on Computer Vision (ECCV), pp. 740–755 (2014)
- [47] Recht, B., Roelofs, R., Schmidt, L., Shankar, V.: Do ImageNet classifiers generalize to ImageNet? In: Proceedings of the 36th International Conference on Machine Learning (ICML), pp. 5389–5400 (2019)
- [48] NVIDIA: TensorRT Model Optimizer. <https://github.com/NVIDIA/TensorRT-Model-Optimizer>. Version 0.47 (2026)
- [49] Migacz, S.: 8-bit Inference with TensorRT. NVIDIA GPU Technology Conference (2017)
- [50] Micikevicius, P., Stosic, D., Burgess, N., Cornea, M., Dubey, P., Grisenthwaite, R., Ha, S., Heinecke, A., Judd, P., Kamalu, J., Mellempudi, N., Oberman, S., Shoeybi, M., Siu, M., Wu, H.: FP8 formats for deep learning. arXiv preprint arXiv:2209.05433 (2022)
- [51] Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., Wu, H.: Mixed precision training. In: International Conference on Learning Representations (ICLR) (2018)
- [52] NVIDIA: NVIDIA H200 Tensor Core GPU Datasheet. <https://resources.nvidia.com/en-us-data-center-overview-mc/en-us-data-center-overview/hpc-datasheet-sc23-h200> (2023)
- [53] NVIDIA: Working with Quantized Types – NVIDIA TensorRT Developer Guide. <https://docs.nvidia.com/deeplearning/tensorrt/10.x.x/inference-library/work-quantized-types.html> (2026)
- [54] Padilla, R., Passos, W.L., Dias, T.L.B., Netto, S.L., Silva, E.A.B.: A comparative analysis of object detection metrics with a companion open-source toolkit. Electronics **10**(3), 279 (2021) <https://doi.org/10.3390/electronics10030279>

- [55] Yang, Z., Adamek, K., Armour, W.: Accurate and convenient energy measurements for GPUs: A detailed study of NVIDIA GPU’s built-in power sensor. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC24) (2024). <https://doi.org/10.1109/SC41406.2024.00028>
- [56] Hendrycks, D., Dietterich, T.: Benchmarking neural network robustness to common corruptions and perturbations. In: International Conference on Learning Representations (ICLR) (2019)
- [57] Eurostat: Non-household electricity prices in 2nd half of 2025. Eurostat news release, 8 May 2026, <https://ec.europa.eu/eurostat/web/products-eurostat-news/w/ddn-20260508-2> (2026)
- [58] Ember: European Electricity Review 2024. <https://ember-energy.org/latest-insights/european-electricity-review-2024/> (2024)
- [59] Wang, C.-Y., Bochkovskiy, A., Liao, H.-Y.M.: YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 7464–7475 (2023)
- [60] Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., Chen, J.: DETRs beat YOLOs on real-time object detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16965–16974 (2024)
- [61] TorchVision maintainers and contributors: TorchVision: PyTorch’s Computer Vision Library. <https://github.com/pytorch/vision> (2016)
- [62] Taori, R., Dave, A., Shankar, V., Carlini, N., Recht, B., Schmidt, L.: Measuring robustness to natural distribution shifts in image classification. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 18583–18599 (2020)
- [63] Rouhani, B.D., Zhao, R., More, A., Hall, M., Khodamoradi, A., et al.: Microscaling Data Formats for Deep Learning (2023). <https://doi.org/10.48550/arXiv.2310.10537>
- [64] Geiger, A., Lenz, P., Urtasun, R.: Are we ready for autonomous driving? The KITTI vision benchmark suite. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 3354–3361 (2012). <https://doi.org/10.1109/CVPR.2012.6248074>